Apache IoTDB: A Time Series Database for Large Scale IoT Applications

CHEN WANG, Tsinghua University, Beijing, China
JIALIN QIAO, Timecho Ltd, Beijing, China
XIANGDONG HUANG, Tsinghua University, Beijing, China
SHAOXU SONG, Tsinghua University, Beijing, China
HAONAN HOU, Timecho Ltd, Beijing, China
TIAN JIANG, Tsinghua University, Beijing, China
LEI RUI, Tsinghua University, Beijing, China
JIANMIN WANG, Tsinghua University, Beijing, China
JIAGUANG SUN, Tsinghua University, Beijing, China

A typical industrial scenario encounters thousands of devices with millions of sensors, consistently generating billions of data points. It poses new requirements of time series data management, not well addressed in existing solutions, including (1) device-defined ever-evolving schema, (2) mostly periodical data collection, (3) strongly correlated series, (4) variously delayed data arrival, and (5) highly concurrent data ingestion. In this paper, we present a time series database management system, Apache IoTDB. It consists of (i) a time series native file format, TsFile, with specially designed data encoding, and (ii) an IoTDB engine for efficiently handling delayed data arrivals and processing queries. We introduce a native distributed solution with distributed queries optimized by parallel operators. We also explore efficient TsFile synchronization mechanisms, ensuring seamless data integration without the need for ETL processes. The system achieves a throughput of 10 million inserted values per second. Queries such as 1-day data selection of 0.1 million points and 3-year data aggregation over 10 million points can be processed in 100 ms. Comparisons with InfluxDB, TimescaleDB, KairosDB, Parquet and ORC over real world data loads demonstrate the superiority of IoTDB and TsFile.

CCS Concepts: • **Information systems → Data management systems;**

Additional Key Words and Phrases: Time series, data model, database engine, distributed

This work is supported in part by the National Natural Science Foundation of China (92267203, 62021002, 62072265, 62232005), the National Key Research and Development Plan (2021YFB3300500), the Beijing National Research Center for Information Science and Technology (BNR2025RC01011), 31511130201, and Alibaba Group through Alibaba Innovative Research (AIR) Program.

Authors' Contact Information: Chen Wang, Tsinghua University, Beijing, Beijing, China; e-mail: wang_chen@tsinghua.edu.cn; Jialin Qiao, Timecho Ltd, Beijing, Beijing, China; e-mail: jialin.qiao@timecho.com; Xiangdong Huang, Tsinghua University, Beijing, Beijing, China; e-mail: hxd@timecho.com; Shaoxu Song (Corresponding author), Tsinghua University, Beijing, Beijing, China; e-mail: sxsong@tsinghua.edu.cn; Haonan Hou, Timecho Ltd, Beijing, Beijing, China; e-mail: haonan.hou@timecho.com; Tian Jiang, Tsinghua University, Beijing, Beijing, China; e-mail: jiangtia18@mails.tsinghua.edu.cn; Lei Rui, Tsinghua University, Beijing, Beijing, China; e-mail: rl18@mails.tsinghua.edu.cn; Jianmin Wang (Corresponding author), Tsinghua University, Beijing, Beijing, China; e-mail: jimwang@tsinghua.edu.cn; Jiaguang Sun, Tsinghua University, Beijing, Beijing, China; e-mail: sunjg@tsinghua.edu.cn.



This work is licensed under a Creative Commons Attribution-NoDerivatives 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 0362-5915/2025/05-ART7
https://doi.org/10.1145/3726523

ACM Reference Format:
Chen Wang, Jialin Qiao, Xiangdong Huang, Shaoxu Song, Haonan Hou, Tian Jiang, Lei Rui, Jianmin Wang, and Jiaguang Sun. 2025. Apache IoTDB: A Time Series Database for Large Scale IoT Applications. *ACM Trans. Datab. Syst.* 50, 2, Article 7 (May 2025), 45 pages. https://doi.org/10.1145/3726523

1 Introduction

In the **Internet of Things (IoT)**, a huge amount of time series is generated by various devices with many sensors attached. The data need to be managed not only in the cloud for intelligent analysis but also at the edge for real-time control. For example, more than 20,000 excavators are managed by one of our industrial partners, a maintenance service provider of heavy industry machines, each of which has hundreds of sensors, e.g., monitoring engine rotation speed. As illustrated in Figure 1, the data are first packed in devices and sent to the server via a 5G mobile network. In the server, the data are written to a time series database for **Online Transaction Processing (OLTP)** queries. Finally, data scientists may load data from the database to a big data platform for complex analysis and forecasting, i.e., **Online Analytical Processing (OLAP)** tasks.

1.1 IoT Scenarios

The process in Figure 1 poses new requirements to time series database management systems. (1) In the end device, such as the aforesaid excavator, a lightweight database or a compact file format is needed to save space and network bandwidth. (2) In the edge server, a full-function database collects, stores and queries the massive data of devices, capable of handling delayed arrivals. (3) In the cloud, a database cluster with complete historical data persistence connects directly to big data analysis systems, such as Spark and Hadoop, and enables OLAP queries. In addition to the large scale issues, millions of series (columns) and billions of points (rows), we highlight below the unique and urgent features in the IoT scenarios.

1.1.1 Device-defined Ever-evolving Schema. Unlike the traditional relational databases with predefined schema, the schema of time series data in the IoT scenario is defined by sensors in the devices. During the device maintenance or upgrade, sensors are frequently removed, replaced or augmented, leading to the changed schema. For instance, as illustrated in Figure 2, sensor FC32 for monitoring fuel consumption is replaced by FC3X, at time 09:06:13. We need a data model that is sufficiently flexible to capture such an ever-evolving schema.

1.1.2 Mostly Periodical Data Collection. Machine-generated sensor data are often collected periodically with a pre-set frequency. While the time series is expected with a regular time interval, there may be small variations due to data bus congestion or network delay. Even worse, those values not changed with the previous may be omitted to save energy. For example, in Figure 2, ES05 is mostly collected every 60 seconds, but with a small delay from time 09:04:13 to 09:04:20 and an omitted data at time 09:07:13. Data encoding should be able to handle such variations for efficient storage.

1.1.3 Strongly Correlated Series. It is also worth noting that multiple sensors, e.g., in the same module of a device, may collect data at the same time. In addition to the same timestamps, their values may also be correlated. For instance, in Figure 2, the fuel consumption (FC32/FC3X) value should be determined by engine speed (ES05) and torque (ET03) at the same time. Moreover, the wind speeds of close turbines in the same wind farm should be similar with each other. Again, the storage scheme is expected to fully utilize such opportunities in data compression.

1.1.4 Variously Delayed Data Arrival. While most data points arrive in time order, serious delays may occur, e.g., owing to network delay or corruption. For instance, in Figure 2, the data

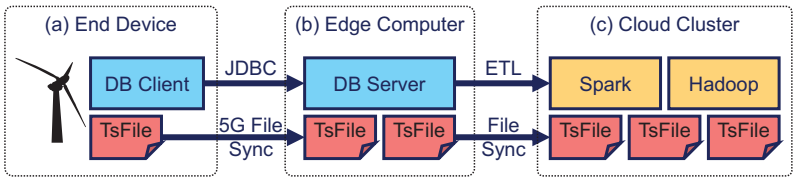


Fig. 1. Data management in IoT scenarios.

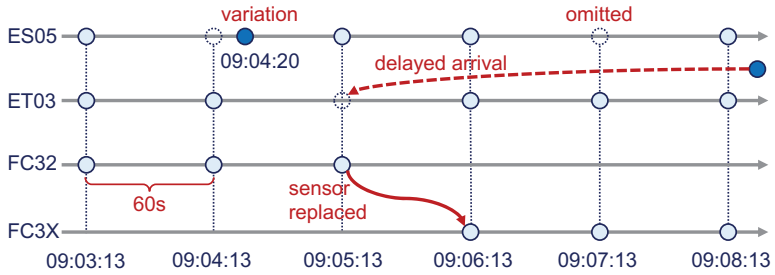


Fig. 2. Time series management issues in IoT scenarios.

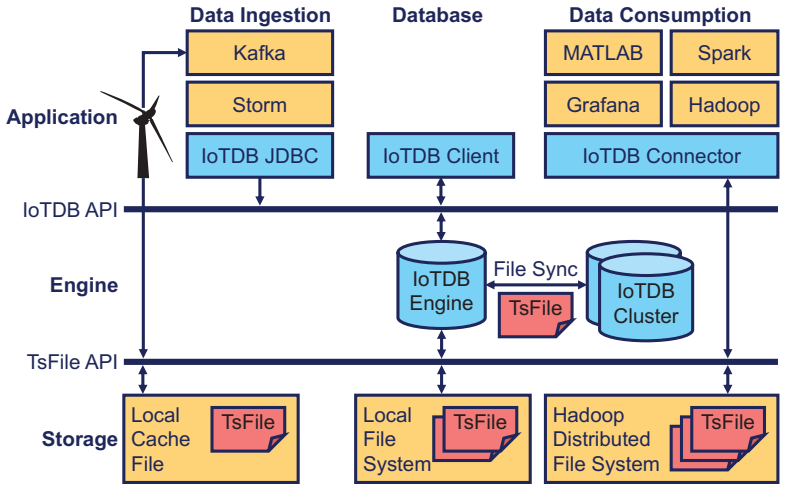


Fig. 3. A data life cycle view of the IoTDB system.

point with timestamp 09:05:13 arrives after it subsequent points. The delay could be various, ranging from seconds to days. This issue is unique in time series data and seriously obstruct the time ordered storage.

1.1.5 Highly Concurrent Data Ingestion. High concurrency is prevalent in the IoT scenarios, from edge devices to the cluster. For instance, each wind turbine has 500+ sensors generating data at the same time. The database needs to ingest the data of 100-200 turbines in a wind farm, while the cloud cluster manages 20,000+ turbines in total. The highly concurrent data make not only database ingestion challenging, but also the replication among nodes in the cluster.

In this study, we introduce a novel, IoT-oriented time series database, IoTDB, built upon a new columnar time series file format, TsFile. As illustrated in Figure 3, the solution is deployed ranging from end devices for data collection, edge servers for local control, to the cloud for device

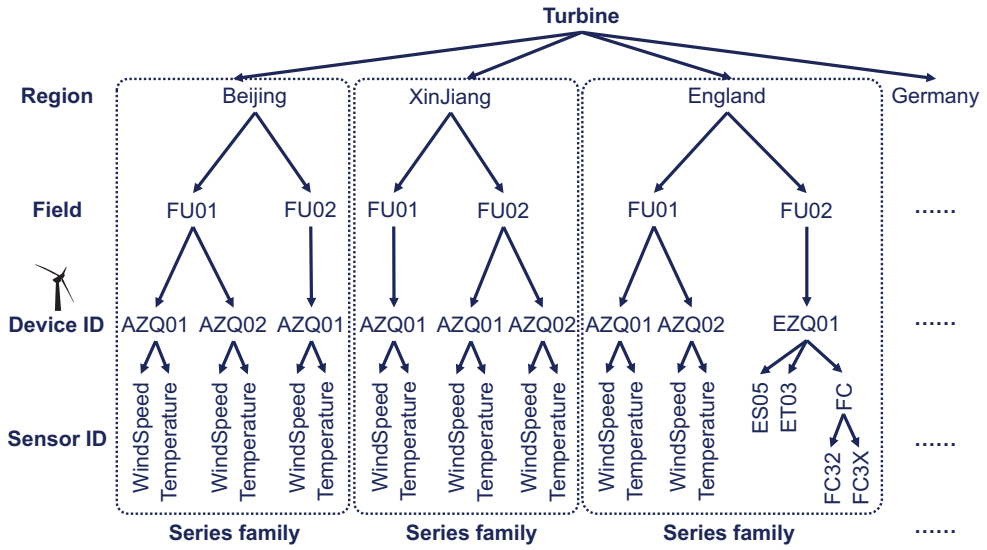


Fig. 4. Data model in IoTDB in the form of a tree structure.

monitoring and advanced analysis. It manages millions of time series, stores terabytes of data, inserts 10 million points per second, selects 0.1 million points and aggregates 10 million points in 100 milliseconds. The open architecture enables both real-time query and complex data analysis over the same data without ETL. (Please see the system design in Section 2.) In general, the features unique in IoT scenarios, aforementioned in Section 1.1, lead to our design choices and novelty below.

1.2 TsFile Format

To avoid the costly ETL process, the same TsFile could be deployed and synchronized among end devices, edge servers, and cloud nodes, without packing and unpacking.

1.2.1 Design Choices. The major features of TsFile, introduced in Section 4, are highlighted as follows. (1) An IoT native data model organizes time series in a tree structure of devices and sensors. The tree structure could naturally represent the evolving schema, e.g., FC32 and FC3X as siblings in Figure 4 both measuring the fuel consumption before and after sensor replacement. It also serves as an index on series for fast access. (2) Columnar storage of time series with time and value columns enables efficient data encoding specially designed for regular time intervals with variations. Moreover, by packing multiple devices and sensors together in one file, the same time column could be shared among multiple series with aligned timestamps. It also enables effective value encoding by leveraging the correlations among series.

1.2.2 Novel Techniques.

Automatic Schema Identification. While the series tree structure is capable of representing the ever-evolving schema, we also propose methods [63] to automatically identify the changed sensors monitoring the same thing, such as the aforesaid FC32 and FC3X both measuring the fuel consumption before and after sensor replacement. Please see Section 3.3 for an introduction.

Encoding Techniques. Moreover, our comparative analysis of encoding [69] shows that most existing data encoding methods do not consider the unique features of time series in the IoT scenario. For timestamp encoding, we propose to imply the regular time interval and thus only need to store

Table 1. File Comparison

Dimension	TsFile	Parquet	ORC
Data model	Series tree	Nested	Tabular
Index on seriesID	Yes	No	No
Time encoding	Regular interval	Long-typed	Long-typed
Value encoding	Multi-series	Nested-column	Single-column
Frequency domain	Yes	No	No

the small variation [23], e.g., 09:04:20-09:04:13=7 in Figure 2. Note that different sensor modules or devices may collect data at the same time with identical frequency, sharing the same timestamps. Thereby, we further propose to align the time series for efficient storage by eliminating their similar timestamps [22]. For value encoding, we obtain a regression model to capture the correlations among series and store only the difference between the regression and the observation values [24]. Please see Section 4.3 for more details.

Frequency Domain Analysis. Frequency domain analysis on time series is widely conducted. While it is costly to online transform from time domain to frequency domain, e.g., by **Fast Fourier Transform (FFT)**, we store the frequency domain data for reuse, via efficient data encoding in TsFile [68]. Please see Section 4.4 for a discussion.

Aligning Multi-variate Time Series. The challenge of multi-variate time series storage is that different time series may have redundant information, leading to wasted storage. While aligning time series can reduce the duplicated timestamps, the difficulty is how to further eliminate the redundant information among values. Compared with the time series alignment in the previous work [24], the value correlation of various time series needs to be considered to further compress the storage cost.

1.2.3 Comparison. As illustrated in Table 1, the aforesaid features unique in IoT scenarios are not considered in Parquet [51] or ORC [29] for nested or tabular data. Given the time series specialized index and encoding, extensive experiments in Section 8.2 demonstrate the superiority of TsFile against Parquet and ORC.

1.3 IoTDB Engine

Over the same TsFile, the IoTDB engine could handle extremely intensive writes as well as various read loads including real-time queries (OLTP) and complex data analysis (OLAP).

1.3.1 Design Choices. The key design choices are discussed in Section 5 and summarized below. (1) An IoT adapted LSM-tree, Log-Structured Merge-Tree [46], efficiently handles delayed data arrivals in a write-intensive workload. For the points with a short delay, the data are first cached in MemTables and then flushed to disk as TsFile sorted on time. A pipeline of sorting, encoding and compressing is designed to enable parallel computing. (2) A query engine fully utilizes the statistics maintained in different levels of TsFile. Note that the data flushed in TsFile are immutable in the LSM-tree data organization. It motivates us to pre-compute the statistics in different levels of TsFile and utilize them in query optimization, in particular in answering aggregation queries.

1.3.2 Novel Techniques.

Automatic Delayed Point Separation. For the points with a long delay, i.e., their subsequent points have already been flushed to TsFile, they will be written to other TsFiles. It leads to the overlapping time ranges of different TsFiles, incurring great query overhead in the LSM-tree like

Table 2. DBMS Comparison

Dimension	Apache IoTDB	InfluxDB	TimescaleDB	KairosDB
Storage	TsFile	TSM	PostgreSQL	Cassandra
Statistics	Yes	No	Yes	No
Organization	LSM (pipelined)	LSM	B+tree	LSM
Data delay	Separation	Merge	Merge	Merge
Replication	NB-Raft	HH	Streaming	HH

data organization. According to our study in many real applications [40], the long delay does not occur frequently, e.g., only 0.0375% in the aforesaid heavy machine service provider. In this sense, we propose to automatically separate the delayed data points to reduce the overlaps of TsFiles for efficient query processing [40]. Please see Section 5.3 for more details.

Efficient Backward Sorting. The challenge of sorting is to execute fast enough so that data ingestion and other tasks will not be blocked. While the backward-sort successfully utilize the delay features of IoT data to accelerate the algorithm, it is still difficult to sort large temporary memory blocks. Compared with the backward-sort in the previous work [74], the extension of sorting in parallel is needed which can still fully investigate the delay features of IoT data.

Machine Learning Queries. The challenge of machine learning is that the data often need to be entirely extracted out of the database. The difficulty of efficient in-database ML is that the data are stored in out-of-order manner in the LSM-Tree, blocking the pre-computation. While a simple AR model is supported in the previous work [62], the more effective ARIMA extension is needed to enhance the in-database applications of IoTDB, such as online anomaly detection, classification and forecasting.

Distributed Query Processing. The challenge of distributed query processing is the potential bottleneck on any single node owing to the uneven load. While the non-parallel operators are easy to schedule and execute, the difficulty is that a long query operation may block the distributed query processing. Compared with the serial operators in the preliminary version [67], the parallel operator extension is needed to accelerate the distributed query processing.

Advanced File Synchronization. The challenge of synchronization among replicas is that it should not occupy too many resources, hindering normal data ingestion. While file-based synchronization can largely reduce the replication messages, the difficulty is that raw files are large and contain redundant information, wasting bandwidth significantly. Compared to the file-based synchronization in the preliminary version [67], the fine-grained chunk-based synchronization extension is needed to reduce the redundant information.

Non-Blocking Raft. To replicate data in a cluster, consensus protocol such as Raft [47] is employed. However, the serialization of appending entries blocks the subsequent requests and thus limits the parallelism and throughput of the system. We propose NB-Raft [37], non-blocking replication for highly concurrent IoT data. In this way, more requests can be processed in parallel, and thus the throughput increases, essential for IoT applications often with vast sensors and fast data ingestion. Please see Section 6.2 for an introduction.

Dynamic Read Consistency Level. For inconsistent replicas, rather than waiting for all the replicas to respond, we propose a DYNAMIC read consistency level [64], returning results based on a part of the replicas responded thus far, as introduced in Section 6.4.

1.3.3 Comparison. As illustrated in Table 2, NoSQL systems such as HBase [5], Cassandra [41] and Druid [70] as well as their time series adaptions OpenTSDB [48] built on HBase and KairosDB [38] on Cassandra, do not have special consideration of variously delayed data arrivals, unique in time series data. It is not addressed either in TimescaleDB [65] built on RDBMS PostgreSQL or the time series specialized InfluxDB [30]. Again, the experiments in Section 8.3 show that IoTDB outperforms the aforesaid InfluxDB, TimescaleDB, KairosDB, and so on.

1.3.4 Open Source. IoTDB is now an Apache Top-Level Project [6]. The system is well integrated with the open-source ecosystem, such as JDBC driver, Kafka [10] for data ingestion, Grafana [15] for data visualization, Hadoop [4] and Spark [72] for data analysis, and so on.

1.4 Paper Organization

The whole paper is organized as follows. In Section 2, we provide the overview of the system design. Section 3 presents the data model of time series, upon which the following techniques are developed. Then, the data file format, namely TsFile, is designed in Section 4. On the top of TsFile, we detail the database engines of IoTDB in Section 5. By running database engines on multiple nodes, we form a cluster in Section 6. Across multiple clusters, the data is shared via file synchronization, which is detailed in Section 7. Then, all designs and techniques are evaluated in Section 8. Finally, Section 9 discusses related work, and Section 10 concludes the paper.

2 System Design

Figure 3 presents an overview of the IoTDB system. In contrast to OpenTSDB [48] built on HBase and KairosDB [38] on Cassandra, IoTDB is constructed from scratch. Its core components (in blue and red) successfully fulfill the requirements in the whole life cycle of time series data in IoT applications as follows.

Data Ingestion and Synchronization. To ingest data, as illustrated in Figure 3, the most direct way is to generate the TsFile, and periodically synchronize the written TsFile to the edge server. Alternatively, one may also use IoTDB JDBC to write the data in real time through the IoTDB engine. For extremely heavy writes, big data processing systems such as Kafka [10] and Storm [11] could be connected to absorb burst traffic.

In some industrial scenarios, there may be many IoTDB instances. For example, a company with many wind farms deploys an IoTDB instance in each wind farm for easier local control of its managed wind turbines. To exchange data between wind farms and converge to the data center, one IoTDB instance synchronizes its local TsFiles to another IoTDB instance. The remote IoTDB receives TsFiles and loads them automatically. In this way, we bridge the gap between end devices and edge servers as well as cloud nodes.

Database Engine. **Time Series File (TsFile)** is a new file format designed to efficiently manage the time series data, presented in Section 4. They can be resident in either the local file system of end devices and edge servers, or the **Hadoop Distributed File System (HDFS)** [4] in a cluster. TsFiles can be directly accessed by the IoTDB engine for query and by Hadoop/Spark through the TsFile Connector for analysis. By this means, we support both OLTP and OLAP without reloading data to different stores.

The IoTDB Engine is the core part of IoTDB, introduced in Section 5. It contains a storage engine that can manage millions of time series and write millions of data points per second (with potentially delayed arrivals). It also contains a query engine optimized to get results in a few milliseconds over trillions of data points. Advanced queries on time series supported in IoTDB include but are not limited to downsampling, aggregation, pattern matching, and most importantly are

extensible through the **User-Defined Functions (UDF)**. The IoTDB Client provides a command-line interface with features like continuous queries and triggers, supporting local monitoring and control on edge servers (e.g., in wind farms). A distributed solution of IoTDB is also introduced in Section 6.

Data Consumption. With various built-in (and future extensible) connectors, the data in IoTDB could be easily consumed by downstream applications. For instance, with the Grafana-IoTDB connector, users can design their dashboard in Grafana [15] to visualize the time series. The IoTDB Client also enables users to write SQL queries and visualize the data interactively in Apache Zeppelin [12]. Using MATLAB, one can read time series data from IoTDB and execute mathematical calculations for analysis. Through the Hadoop-IoTDB and Spark-IoTDB connectors, long historical data can be processed and analyzed in parallel by MapReduce.

3 Data Model

Section 2 gives an overview of the system design. Before introducing each system component in the design, it is necessary to declare the data model of IoT data. Data model is indeed the foundation of developing all the system components in the following sections.

While InfluxDB [30] uses tags to organize and identify time series, we notice after a long survey in our many industrial partners that tags only are too weak to manage devices and sensors rigidly in enterprises, and difficult to optimize the physical storage.

3.1 Design: Logical Schema

IoTDB manages all time series in the form of a tree structure, where each leaf node corresponds to a time series. The hierarchical structure naturally fits the management levels in industry [54] and thus helps users organize time series methodically. An example of the tree in the wind turbine scenario is shown in Figure 4. In particular, the following two levels appear almost in all IoT applications.

In Figure 4, Field means the specific location of a wind farm, analogous to a district or a block in a city. Different values are stored at the leaves in the right sub-tree rooted at England, since EZQ01 is a different type of wind turbine with various sensors. This subtree is deeper than the others. The reason is that as introduced in Section 1.1.1, sensor FC32 is replaced by FC3X at some time. Both sensors measure auxiliary fuel consumption and thus are the children of FC. A typical wind turbine usually contains devices like blades, wheel hubs, motors, and so on, with sensors measuring wind speed, temperature, voltage, and the like.

Sensor. A sensor measures a physical quantity such as power, voltage, current, bracket displacement, wind speed, vehicle speed, longitude, latitude, and so on. As shown in Figure 4, each leaf node of the tree corresponds to a sensor. Due to space limitations, only a part of the sensors is plotted in this figure.

Device. A device is a piece of equipment that has sensors. In IoTDB, each sensor has its corresponding owning device. As shown in Figure 4, a device is located by the path from the root node to the penultimate level node.

3.2 Design: Physical Schema

While the hierarchy describes how the time series are organized at the logical level, the physical schema hidden beneath further specifies the organization in the physical storage. Although a tree could be unbalanced in the logical design, it does not affect the grouping of time series in sub-trees

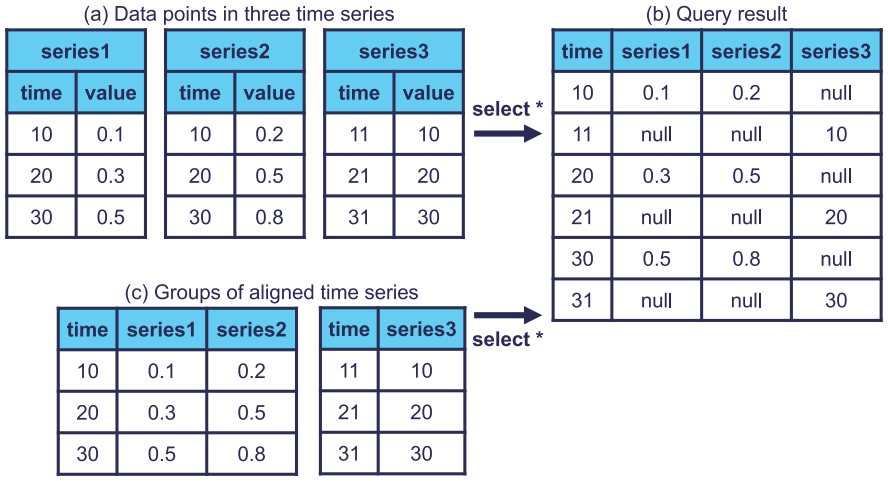


Fig. 5. Aligned query result of time series data. The same results can be produced by aligned time series in (c) without storing duplicated timestamps.

in the physical design. For example, as illustrated in Figure 4, the time series identified by paths "Turbine.England.FU01.AZQ01.WindSpeed" and "Turbine.England.FU02.EZQ01.FC.FC32" with different lengths, respectively, belong to the same sub-tree of England. They can be physically stored together in the same series family (analogous to a database instance).

3.2.1 Time Series. A time series is a series of data points recorded by a sensor over time. Each series consists of two attributes, time and value, as presented in Figure 5(a). A time series is located by the full path from the root node to the leaf node. For example, in Figure 4, the path from the root Turbine.Beijing.FU01.AZQ01.WindSpeed denotes the time series recorded by the wind speed sensor of the AZQ01 device in the FU01 field of Beijing.

3.2.2 Series Family. Time series from the same device are stored together in TsFiles and a TsFile can store time series from multiple devices. We further control over which devices are stored together based on application requirements, which is the motivation behind the concept of the series family. A series family contains multiple devices whose data will be stored together in TsFiles. Each series family has an independent storage engine, and all TsFiles in it will be stored in the same directory. Thereby, series families are physically isolated like the "database" concept in traditional databases, physically separating the data of different users and meanwhile facilitating data migration and analysis. In Figure 4, all the time series in one region form a series family.

3.3 Technique: Automatic Schema Identification

Different from the traditional schema-first databases, the data may come before the corresponding schema defined in the IoT scenarios, i.e., data-first. For instance, a wind turbine immediately starts to transfer data when installed before registering it in the data center. Moreover, workers may occasionally mis-plug the cables of sensors during equipment maintenance, misplacement among different sensors occurs frequently. According to our preliminary study [63], 200 out of 5.2k tuples are observed with scheme issues in the real wind turbine data. In this sense, we offer automatic schema identification of time series [63]. It enables users to insert data without defining the schema in advance, highly demanded in the write-intensive IoT scenarios to avoid data loss.

3.4 Technique: Aligning Multi-variate Time Series

By default the time series is stored as uni-variate, as illustrated in Figure 5(a), where each series corresponds to a pair of time and value sequences. Note that in practice some series of the same device may collect data at the same time, e.g., series1 and series2 in Figure 5(a) share the same timestamps of 10, 20, 30, ... It is highly practical to store the aligned time series by eliminating the duplicated time columns, known as multi-variate time series. For instance, Figure 5(c) shows the group of series1 and series2 by sharing the same column of timestamps.

The question is thus how to efficiently identify the groups of series that share similar timestamps. Note that by further aligning series3 into the group of series1 and series2, the aligned multi-variate time series has many null values, as in Figure 5(b), leading to waste of space. Unfortunately, it is proved NP-hard to find the optimal column groups for the minimum space cost according to the analysis, and thereby a heuristic algorithm is devised for automatically grouping time series for efficient columnar storage [22].

4 TsFile Format

According to both the logical and physical models in Section 3, we design a special file format for time series data, namely TsFile. The two-level physical model is mapped onto Chunk and Chunk Group in the data part of a TsFile, which will be introduced in Section 4.1. For the unbalanced logical model, a tree-like index is built within the index part of a TsFile. Then, the columnar storage of single series is introduced in Section 4.2.

To satisfy the IoT requirements and offer fast data ingestion, efficient query processing, and compact storage space at the same time, we present the *Time Series File* (TsFile), the first file format optimized for time series data management.

4.1 Design: File Structure

When designing the structure of TsFile, we recognize that (1) the data should be compressed as much as possible for saving space; (2) multiple devices need to be packed together to reduce the number of files; (3) some time series are often queried together and expected to be close in physical locations; (4) the data ought to be packed with sizes coincident with file systems to avoid disk fragments; and (5) millions of time series need to be efficiently accessed. Figure 6 shows the structure of a TsFile.

Page. A page is a basic unit to store time series data on disks. Each time series in a page is sorted by time in ascending order, with two columns, one for timestamps and the other for values. The timestamps and values are stored and compressed separately. Such a columnar store compresses time series data greatly, given the increasing time and steady value in many industrial applications (see a detailed discussion in Section 4.2). While pages share the same size for easier access, the amount of data on a page is configurable, affecting the performances of compression and query processing.

Chunk. A chunk consists of a metadata header and several pages, all belonging to one time series. Unlike pages, the chunk size is variable. The chunk metadata includes the name of a series, the data type of this series, as well as the encoding and compression information, which allows us to use different algorithms to compress different time series. Such information of a series is also called the schema of a series.

Chunk Group. A chunk group contains multiple chunks, which belong to one or multiple series of a device written in the same period of time. The reason for grouping them together in the continuous disk space is that these chunks of series of a device are often queried together. As

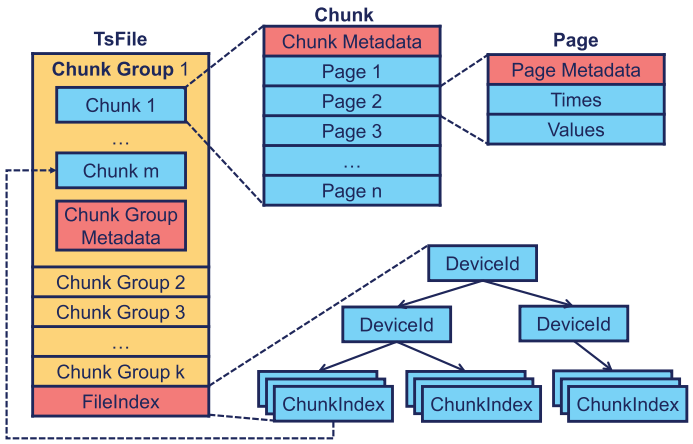


Fig. 6. TsFile structure optimized for time series data management in the IoT hierarchy.

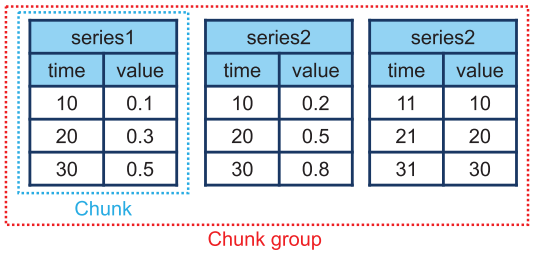


Fig. 7. Time series data organized by chunk and chunk group.

shown in Figure 7, a chunk stores some sorted, encoded, and compressed data points of a time series, while a chunk group consists of (three) chunks of the same device.

Block. When writing data to a TsFile, all chunk groups being written are buffered in memory first. We bound the memory consumption by setting a threshold on the total memory usage of all active chunk groups. When the memory usage reaches the threshold, we flush all chunk groups to the TsFile. Therefore, these chunk groups form a block, which is a logical concept. The block size can be adjusted for achieving data locality in HDFS.

Index. The file metadata at the end of TsFile contains a chunk-level index and the file-level statistics for efficient data access. File-level statistics record the statistics of each time series, such as start time, end time, count, and the corresponding positions of the chunk-level index. The chunk-level index records the position of each chunk and is grouped by device and sensor. That is, we can directly get all chunk metadata belonging to a time series. Each index also contains the time information, such as the maximum and minimum timestamps, which can be used in query optimization.

4.2 Design: Columnar Storage

Time and Value Columns. As illustrated in Figures 5(a) and 6, we store the time column for each time series instead of storing only one time column in a device or a data file, which is a tradeoff between space and time. The design choice is made for the following benefits. (1) Null value elimination. There is no need to store null values for aligning multiple series, which saves

the disk occupation and simplifies the implementation. (2) Data access locality. Each time series can be directly processed inside pages, without the need to join the time column elsewhere.

In most cases, sensors collect data at a regular time interval. However, the timestamps usually have small deviations in the real world, which result in many null values when aligning multiple time series, wasting space. Indeed, the time column can be compressed concisely using a proper compression method as introduced below. Therefore, though the data access may introduce the extra overhead of aligning multiple series, this tradeoff is worthy.

Encoding and Compression. By separating the timestamp and value into individual columns, we can adopt different encoding and compression methods for time and value columns. In TsFile, we only compress data at the page level so that we do not need to uncompress the whole file to read a part of the data.

We first translate timestamps/values into bytes by data encoding, then use a compression method to compress the encoded bytes. Many existing encoding methods are implemented, including **run-length encoding (RLE)** [26], Bit-packing [18], delta-delta encoding (2DIFF) [3], GORILLA [52] and so on. The implemented compression methods are Snappy [27], GZIP [20], LZ4 [17], and so on.

4.3 Technique: Times Series Encoding

While there exist many encoding algorithms, our comparative analysis of encoding [69] shows that most of them do not consider the unique features of time series in the IoT scenario.

For timestamp encoding, as introduced in Section 1.1.2, most sensor data are collected with regular time intervals but may have some variation and omission. Therefore, we propose to imply the regular time interval [23], e.g., in every 60 seconds in Figure 2. Rather than storing the long timestamp (or its delta), we only need to store the count of regular time intervals and the small variation, e.g., 2 regular time intervals and 0 variation for 09:08:13 in ES05.

To overcome the difficulty of eliminating the redundant information among values, we propose to align multivariate time series and consider their correlations for better compression. Inspired by tuple alignment strategies initially proposed for regression tasks [24], we adapt the idea to address the challenges of value encoding for multivariate time series. According to time similarity, we first align multivariate time series. For numeric columns, during alignment, we train a regression model to guide the alignment process. This model uses values from multiple columns as input to predict values in the target column. When encoding the aligned multivariate time series, we handle the timestamp columns by encoding the first column with its raw values, while encoding the remaining timestamp columns as residuals relative to the first. For value columns, we encode the raw values of multiple regressed columns, and encode the target column based on the residuals from the output of the regression model. This technique may reduce the bit-width of data and thus reduce the space cost. Since the alignment is performed in groups, the approach ensures broad coverage. Overall, this correlation-based encoding method can deliver significant benefits in space savings when the data exhibit strong inter-column correlations. The experiments in Figure 18 of Section 8.2.5 show that for datasets with strong time series correlation, the correlation compression saves about 30% space, with almost no extra overhead in write and read.

4.4 Technique: Frequency Domain Encoding

Finally, frequency domain analysis on time series is widely conducted in signal processing. It is costly to online transform from time domain to frequency domain, e.g., by Fast Fourier Transform (FFT), due to its quasilinear time complexity. Therefore, we propose to store the frequency domain data for reuse, via efficient data encoding in TsFile [68]. Quantization is first conducted

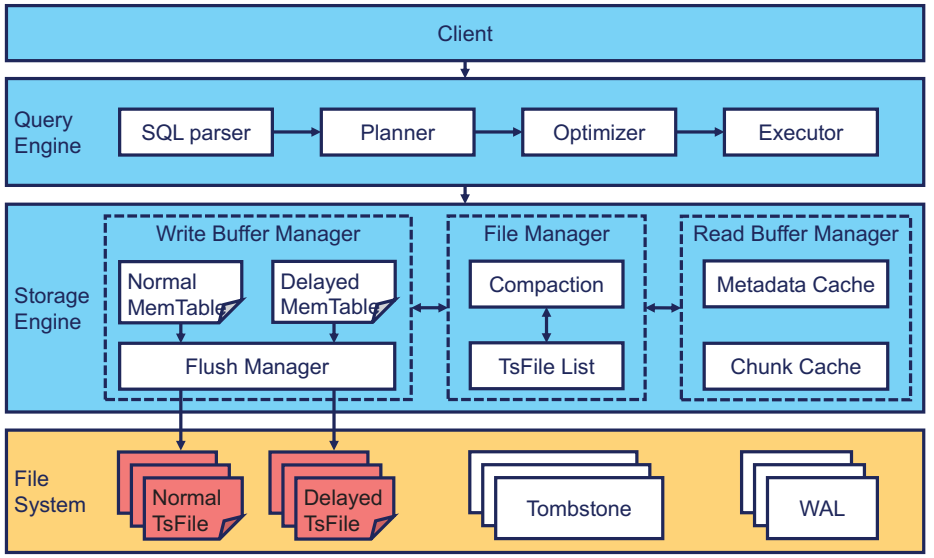


Fig. 8. Architecture of IoTDB engine.

on the transformed frequency domain data, with unnecessarily high precision. Moreover, we devise a bit-width decreasing scheme to further reduce the space, referring to the extremely skewed distribution of frequency domain data.

5 IoTDB Engine

The TsFile introduced in Section 4 can store time series data with high efficiency, but in a system there may be thousands of TsFiles or even more. How to manage massive TsFiles efficiently while keeping the system complexity low is a great challenge. As a solution, we design a storage engine in Section 5.1 and a query engine in Section 5.4, upon the TsFiles. In addition, IoT scenarios have distinct characteristics of queries, often related to time. Thereby, we design special query languages to operate time series data easily, in Section 5.4, while keeping them similar to traditional database queries as much as possible.

The IoTDB engine design fully considers the following IoT scenarios: (1) delayed arrivals, such that data generated first arrive later, (2) efficient query processing, over various time ranges of miscellaneous series, and (3) SQL-like queries, extensive for OLTP and OLAP purposes. Figure 8 presents the architecture of the IoTDB engine.

5.1 Design: Storage Engine

As shown in Figure 8, the storage engine mainly handles the write, read and management of TsFiles. While the read buffer for caching chunks and metadata is standard, we mainly introduce the special design for IoT data writes below. As introduced in Section 1.1.4, the data arrivals could be delayed in various degrees. For example, wind turbines could be occasionally offline for several days, owing to bad weather or network maintenance. When a turbine reconnects, it first reports its current work status to the server and then re-sends data collected in the past several days. The storage engine should be specially designed to handle these delayed arrivals in IoT.

5.1.1 MemTables. To handle intensive writes and delayed arrivals, IoTDB leverages a strategy similar to LSM-Tree [46]. We maintain MemTables in memory for newly inserted data. Different

from a MemTable in KV stores that represents one table, a MemTable in IoTDB contains multiple time series for a period, which need to be sorted separately. Before written to MemTable, each insertion will be added to the **Write Ahead Log (WAL)** to avoid data loss.

For each series family, introduced in Section 3.2.2, we have two active MemTables for data insertion with and without delay, i.e., *delayed* MemTable and *normal* MemTable. To distinguish the delayed data from normal data, we introduce a time detector. It maintains a timestamp for each device, called *stable time*, to record the largest timestamp for each device that has been persistent on disk. For instance, in Figure 10(b), suppose that the point with data time 6 has been written to disk. When IoTDB receives a new data point whose timestamp is greater than the stable time, e.g., the point with data time 7, it is added into the normal MemTable. Otherwise, it is inserted into the delayed MemTable, such as the point with data time 5, arrived at time 7 as well. In this way, it is guaranteed that all the data points in the normal MemTable have time greater than those persistent on disk, i.e., in order.

5.1.2 Flushing. MemTables are flushed one by one to a TsFile for data persistence. When we flush a normal MemTable on disks, the MemTable is appended into an unclosed normal TsFile as a block of chunk groups in Figure 6. Similarly, the delayed MemTable will be written into the unclosed delayed TsFile. The flush tasks are asynchronous and managed by the flush manager.

When the size of an unclosed TsFile exceeds a predefined threshold, or periodically, the file will be closed. Its metadata will be flushed to the end of the file. Then a new unclosed TsFile is created for flushing MemTables.

5.1.3 TsFiles. Referring to the aforesaid stable time, since each *normal* MemTable has data time greater than all the previously flushed normal TsFiles, the data time ranges of all normal TsFiles are non-overlapping, denoted by red rectangles in Figures 10(b) and 10(d). In contrast, *delayed* MemTable contains data points with time smaller than the stable time, i.e., the flushed delay TsFiles, denoted by the blue rectangles in Figures 10(b) and 10(d), may have time ranges overlapping with the other normal/delayed TsFiles.

Normal TsFiles of a series family form a list ordered by the data timestamp, while delayed TsFiles are in the other list sorted by the ingestion time, a.k.a. version. Two lists in a series family are maintained by the file manager for read-write concurrent safety. Later written data always has a larger version, which is indicated by the TsFile generation time and the chunk offset inside TsFile. If a TsFile generation time is larger, then its data is newer. Inside a TsFile, the data is compared by the offset, i.e., a larger offset means newer data.

When updating a single data point, we consider it as an insertion operation. The updated value in the insertion operation will be saved into a delayed TsFile. In the reading process, if there are two versions of values for a data point, the version in the delayed TsFile will be adopted. In this way, the updated data takes effect. If more than one version exists in delayed TsFiles, the latest-inserted one will be applied.

5.1.4 Tombstones. Deletion is also supported, e.g., for releasing space in the edge server [42]. The delete operation is often in the form of

```
delete ts where time <= t,
```

which will delete the data of time series `ts` whose time is less than or equal to `t`.

Deletions are recorded as tombstones outside the TsFile, which is append-only by flushing. Notably, deletions in IoT applications are rare. To store these operations, a tombstone file is attached to each TsFile. When receiving a deletion, it is applied to the MemTables first, and then stored into a tombstone file of each TsFile that contains the data to delete. The function of the tombstone file

is similar to the differential file [58]. These tombstone files are used to re-apply deletions when querying.

5.1.5 Compaction. According to the aforesaid writing process, time ranges of delayed TsFile and normal TsFile may overlap, which leads to merging the data in multiple files in a query. To accelerate query processing, there is a compaction process to reorganize files in the background. All the three kinds of files, normal TsFiles, delayed TsFiles, and tombstone files, are involved in the compaction process, and will be compacted into new normal TsFiles, which have non-overlapping time ranges. After a compaction process, the corresponding delayed TsFile and tombstone files are removed. In this way, only normal TsFiles with non-overlapping time ranges are left eventually, which accelerates the query problem. The compaction can be triggered periodically in practice.

5.2 Technique: Efficient Backward Sort

As introduced in Section 1.1.4, data arrivals are often delayed for various reasons, e.g., as illustrated in Figure 2. On the other hand, the data points are usually stored and queried in data time order. The inconsistency between arrival time order and data time order leads to out-of-order arrivals. For instance, point p_4, which is generated at time 4 (with data timestamp 4), arrives at time 6.5. In contrast, the latter generated point p_5 with data time 5 arrives at time 6, earlier than p_4. It leads to the zigzag order in Figure 9(a), and should be sorted by data time as in Figure 9(b).

Note that data points can only be delayed, but should never arrive early, i.e., a point would not arrive before it is generated. Enlightened by this delay-only property, we propose to efficiently sort the data points by moving backward the delayed data points [74]. For instance, in Figure 9(a), we move the out-of-order arrival p_4 backward, before the latter generated point p_5. It results in the time series in data time order in Figure 9(b).

To overcome the difficulty of time-consuming block sorting, we introduce parallel processing over different blocks. During the sorting process, data is divided into blocks of adaptive size. The preliminary study [74] sorts in all blocks sequentially, which is time-consuming, accounting for over 70%-90% of the total processing time. In this stage, since each block is sorted independently, we propose to sort these blocks in parallel. Note that the overlaps among blocks are often small, referring to the delay features of IoT data. Thereby, the block merge cost takes only a small part, after each block being sorted in parallel. Consequently, the overall sorting performance can be significantly improved by block sorting in parallel. The experiments in Figure 20 of Section 8.3.2 show that IoTDB with backward-parallel optimization improves the backward sort performance by 10-20%.

5.3 Technique: Automatic Delay Separation

While the out-of-order arrivals with short delay can be sorted in MemTable as introduced in Section 5.2, the long delayed ones can only be processed in the latter MemTable and lead to TsFiles with overlapping time ranges. When there are only a small proportion of delayed data arrivals, i.e., most data are in normal TsFile without time range overlapping, the separation of delayed data helps in query processing and write amplification. To merge the delayed TsFile, we only need to consider those with time ranges covering some points in the delayed one. For instance, for the delayed TsFile denoted by the blue rectangle in Figure 10(b), the two data points only need to merge with two left-bottom normal TsFiles in red. In contrast, for the conventional LSM-tree without separation in Figure 10(a), many files are with overlapping data time and need to be merged.

However, for the cases where most data are disordered, i.e., no clear relationship between data time and arrival time as in Figures 10(c) and 10(d), it is more like the conventional LSM-tree

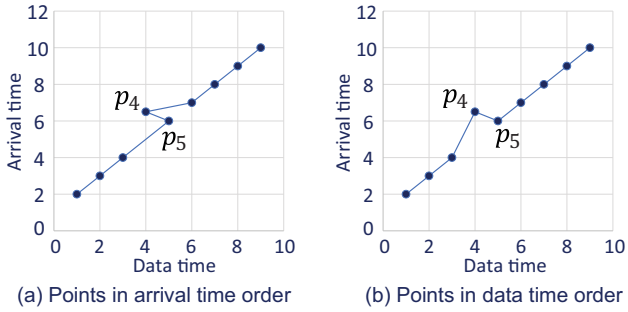


Fig. 9. Time series with out-of-order arrivals, where p_4 is generated earlier than p_5 but arrives later than p_5.

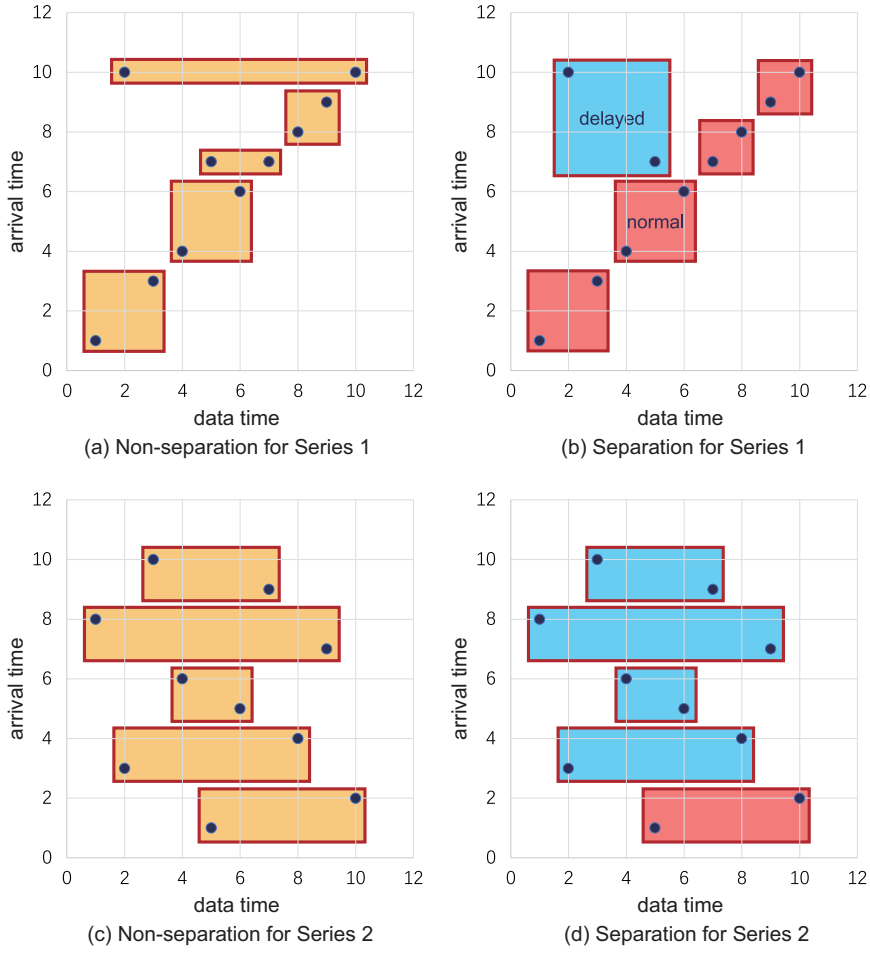


Fig. 10. Separating delays (blue rectangles) works better in (b) than non-separation in (a), but not in (d) compared to (c).

scenario where data arrival is not related to the key (time). In this case, the separation of delayed data has no advantage but incurs the extra cost of maintaining two types of MemTables/TsFiles.

Therefore, we propose to automatically determine whether to separate the delayed data or not, by observing the data arrivals [40]. It is not surprising that both the write amplification and query processing are improved, by avoiding overlapping TsFiles when possible.

5.4 Design: Query Engine

Like typical DBMS, the query engine translates SQL queries into operators that are executable in the database. As illustrated in the architecture in Figure 8, the SQL parser generates an **Abstract Syntax Tree (AST)** of the input SQL from the client. The planner analyzes the AST and generates the query execution plan, which consists of many operators. The optimizer transforms the query execution plan into an optimized form, e.g., by expression compression. The executor receives a query execution plan and conducts the query task to return data. In the following, we introduce the four main query operators in IoTDB.

5.4.1 Selection Operation with Time Predicates. It is common to query a time series with predicates in the time dimension, e.g.,

```
select WindSpeed
from Turbine.Beijing.FU01.AZQ01
where time > 2019-01-01 11:00:00 and time < 2019-02-01 11:00:00.
```

It queries the time series of wind speed within a month. In execution, the query engine builds a series reader to fetch the relevant data from memory and disk, and apply the time predicates on each point to return the query result. When fetching relevant data in memory, the query engine directly extracts the data points of the selected time series that exactly meet the time predicates from MemTables.

To speed up fetching data on disk, the query engine uses different levels of statistics as described in Section 4.1 to filter out irrelevant data at different levels of granularity. Firstly, it uses the file-level statistics of the file metadata, to skip TsFiles that do not contain the selected time series or do not intersect with the query time range. Then, it reads the chunk-level index of the remaining TsFiles, and uses the chunk-level statistics to skip chunks that do not meet the time predicates as a whole. The remaining chunks are read from disk using the chunk position information in the chunk-level index. Finally, the query engine deserializes the page-level statistics and skips deserializing the remaining page data which do not meet the time predicates as well.

5.4.2 Selection Operation with Value Predicates. Predicates on the value dimensions are also widely specified, e.g.,

```
select WindSpeed
from Turbine.Beijing.FU01.AZQ01
where Temperature < 0 and WindSpeed > 0.
```

It queries the time series of wind speed when the temperature is below zero and wind speed is above zero. During query execution, a time generator component is first constructed based on the predicates. The time generator is a binary expression tree, whose leaf nodes are series readers of the relevant time series. The time generator will generate timestamps that satisfy the predicates. The series reader with value predicates works in a similar way as that with time predicates introduced in Section 5.4.1. The only difference is that when value predicates are involved, only statistics of data that do not overlap with any other data on time ranges can be used to help filter out irrelevant data.

Secondly, a series reader is built on the projected time series to retrieve points under those generated timestamps. The query process is optimized in the following two ways. (1) In a series reader, we mark the position of the data point that is currently being read. Then we can get the next data point quickly because the reading process always marches forward in time increasing direction. (2) When a time series in the predicates is also selected, we cache its values under the generated timestamps when the time generator works. The cached data can be used later as the retrieved data for the selected time series, avoiding repeated data retrievals.

5.4.3 Alignment Operation of Multiple Series. When multiple time series are queried, for example,

```
select Temperature, WindSpeed, Voltage
from Turbine.Beijing.FU01.AZQ01
```

a multi-way merging algorithm is applied to join these time series by time. In this example, there will be three series readers built for these time series. Then a merge join is applied upon those series readers. Analogous to outer join in relational databases, if a series does not have a value at a timestamp, a `null` value will be added in the result, as shown in Figure 5.

5.4.4 Aggregation Operation using Statistics. To accelerate the execution of aggregation operators, statistics at different granularities are used, including page, chunk and file statistics, as introduced in Figure 6. Statistics can be directly used to update the aggregation result when the corresponding granularity of data meets the following requirements: (1) all data points satisfy the predicates, and (2) no data point is updated or deleted. During execution, the query engine checks the usability of statistics in descending order of granularity, i.e., the file-level statistics is checked first, then the chunk-level, and finally the page-level. Benefiting from these statistics, it is usually not necessary to retrieve all raw data.

The aggregation query gives a summary of the selected time series by applying an aggregation function on it. For example, the query below gets the number of points in the time series of wind speed from the device `Turbine.Beijing.FU01.AZQ01`:

```
select COUNT(WindSpeed)
from Turbine.Beijing.FU01.AZQ01
where time > t1 and time < t2
```

IoTDB supports multiple aggregation functions such as `max_value`, `min_value`, `last_time`, `first_time`, `count`, `sum`, `avg`, and so on.

An extension of aggregation query called the *sliding window query*, processes a time series in a period of time with a user-defined resolution. For example, the following query gets the average temperature from 00:00 to 08:00 per day in a month:

```
select mean(Temperature)
from Turbine.Beijing.FU01.AZQ01
group by ([2022-01-01, 2022-02-01), 8h, 1d)
```

The process consists of two steps, segmentation and aggregation. To start with, it calculates the first window according to the start time `2022-01-01` and time interval `8h`, i.e., `[2022-01-01 00:00, 2022-01-01 08:00)`. Then, the aggregation of the window is applied. The window will slide by step `1d` and finally return 31 aggregation values to the client.

5.5 Technique: Advanced Extensible Queries

More advanced queries on the time series data for IoT scenarios have been introduced in our preliminary studies and implemented in IoTDB. For instance, the pattern matching query returns

(a) select temperature, voltage, speed
from turbine.Beijing.FU01.AZQ01

time	temperature	voltage	speed
10	0.1	0.2	null
11	null	null	10
20	0.3	0.5	null
21	null	null	20
30	0.5	0.8	null
31	null	null	30

(b) select temperature, voltage, speed
from turbine.Beijing.FU01.AZQ01
fill previous(1min)

time	temperature	voltage	speed
10	0.1	0.2	null
11	0.1	0.2	10
20	0.3	0.5	10
21	0.3	0.5	20
30	0.5	0.8	20
31	0.5	0.8	30

Fig. 11. Fill clause imputes missing values after alignment.

the occurrences that are similar to the input sequence, by a `PATTERN` function. The event pattern query finds events satisfying certain occurrence conditions specified by `ESEQ` and `EAND`. The anomaly detection function identifies outliers as well as the corresponding explanations of why they are outliers . The data imputation function fills the missing or null values, as a `FILL` function.

To meet customized computing needs for domain specific queries, IoTDB allows users to create **User-Defined Functions (UDF)** [9]. It takes one or multiple time series as input, and outputs one time series, which has any number of data points. With UDF, IoTDB provides a variety of built-in functions, such as `SIN`, `LOG10`, and `TIME_DIFFERENCE`. The UDF library [8] now includes more than 60 functions on data profiling, data quality, frequency domain analysis, and so on.

The FILL function is often used for data imputation. Consider the SQL statement below.

```
select temperature, voltage, speed
from turbine.Beijing.FU01.AZQ01
fill previous(1min)
```

The alignment of temperature, voltage, and speed may add nulls, which are filled with the previous values within 1 minute. Of course, the FILL function can be optionally applied. Figure 11 illustrates examples of the alignment operation without and with FILL function. Note that the first null value of speed cannot be filled by this SQL, because there is no previous value.

5.6 Technique: Machine Learning Queries

In-database machine learning is also highly desired, for efficient I/O and computation costs, e.g., learning **Autoregressive (AR)** model over the time series stored in Apache IoTDB [62]. The learning function is again called by SQL statement.

```
select AR(WindSpeed, 'p'='3')
from Turbine.Beijing.FU01.AZQ01
where time >= 2020-01-16 T00:00:00 and time <= 2020-01-16 T23:59:59
```

The query returns the AR model coefficients (with model order p=3), learned from the time series `Turbine.Beijing.FU01.AZQ01.WindSpeed`. The training data are within the specified time range. The learned models can be utilized in downstream applications, such as wind speed forecasting.

It is worth noting that the learning process can be optimized with the database facility. Recall that the LSM-tree based storage engine in Section 5.1 stores time series data in immutable TsFiles with possibly overlapping time ranges. Hence, we may pre-compute the AR model coefficients for the time series in each immutable TsFile. To process the learning query over multiple TsFiles, rather than merging the time series and learning from scratch, we can directly aggregate the aforesaid

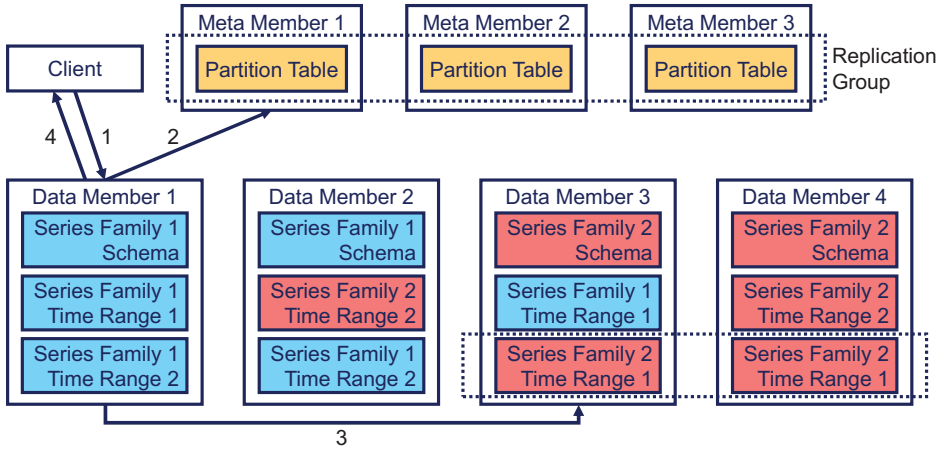


Fig. 12. Architecture of IoTDB cluster.

pre-computed AR models. Given the various scenarios of TsFiles, adjacent, disjoint or overlapped, the corresponding model aggregation methods are designed [62].

To overcome the difficulty of slow out-database ARIMA queries, we introduce LSM-ARIMA, as a new type of optimized machine learning query. Different from accelerating autoregressive models in LSM-AR [62], LSM-ARIMA integrates moving average models by fitting a time series with both AR and MA components simultaneously, enhancing overall fitting effectiveness. It validates that IoTDB can support other types of ML queries like the well-used ARIMA model, apart from the previous AR models. Technically, ARIMA utilizes the same pre-computation as AR models, and obtains the parameters of AR. Then, ARIMA further calculates the residual errors between the AR predicted results and the ground truth, and fits **MA (moving average)** based on these residual errors. That is, our LSM-ARIMA considers AR and MA models simultaneously for an enhanced performance with pre-computation. The experiments in Figure 22 of Section 8.3.4 show one-order-of-magnitude query latency reduction, at the cost of less than 10% write latency increment and not much extra space.

6 IoTDB Cluster

The engines in Section 5 define the behavior of a single IoTDB node. However, it is obvious that a single machine is not qualified to process the big data from IoT scenarios. A cluster of IoTDB nodes must be highly available and efficient, to satisfy the demand of ever-growing IoT data. While TsFiles can be distributed in HDFS and manipulated by Spark as illustrated in Figure 3, we provide a native solution for better data distribution and query processing. Let us first illustrate the overview of the IoTDB cluster architecture in Section 6.1 and distributed query processing in Section 6.3. We then present two optimization strategies in writing and reading replicas, in Sections 6.2 and 6.4, respectively.

6.1 Design: Partition and Replication

Both schema and data are partitioned and replicated, e.g., by consistent hashing, for easier load balance. As shown in Figure 12, the partition table is replicated on all nodes of meta members, which store the hash code of all physical nodes and all series family names.

For better distribution, the schema is partitioned by series family, a group of series often processed together as introduced in Section 3.2.2. The data is also first partitioned by series family

vertically, then by time range horizontally. As illustrated in Figure 12, there are two schema partitions and four data partitions, and each series family corresponds to one schema partition and has two data partitions.

Each schema or data partition is allocated to N nodes by consistent hashing, where N is the replication factor ($N = 2$ in Figure 12). For each partition, the same color rectangles in the dashed box form a replication group. In each replication group, we use a consensus protocol to maintain consistency across multiple replicas. All the partition tables in meta members form a replication group as well.

When writing data into IoTDB, (1) the client sends the request to any data member. (2) The data member gets the partition information from the meta member. (3) The request is then routed to all replicas for data consistency. (4) Finally, the original data member returns the response to the client. The query is processed similarly.

6.2 Technique: NB-Raft for Replication

Consensus protocols are essential in efficiently keeping replicas consistent in distributed systems. Raft consensus protocol [47] models data as a continuous log without holes, making the state space smaller than other protocols, and thus is easier to understand and maintain. Unfortunately, the log serialization scheme of Raft limits its parallelism and consequently the throughput. A typical IoT application manages thousands of devices, samples millions of time series, and generates tens of GBs data every second. Directly applying Raft to IoT scenarios pushes the protocol throughput to its limits, and thus potentially shut out data from the system.

To increase parallelism and throughput, we propose a novel adaption, ***Non-Blocking Raft*** **(NB-Raft)** [37], for high throughput IoT data. Intuitively, once an entry arrives at enough followers (a quorum), rather than waiting for other entries, we may first notify the client so that it can issue the next request. By introducing a new intermediate state WEAK_ACCEPT for entries, the "Early Return" is enabled. Instead of blocking the subsequent entries, multiple entries from the same client can thus be processed in parallel.

6.3 Design: Distributed Query Processing

Distributed queries in IoTDB are implemented by replacing the low-level operators with their associated distributed implementations. Two operators have their distributed implementation: Series Reader and Aggregator. Recall that in Section 5.4, Series Reader scans data of a time series (like a column scan in relational DB), and Aggregator computes the aggregation result of a time series.

By accessing data partition info, a Distributed Series Reader determines in which nodes a time series is located. Then, it constructs a Local Series Reader in each node to read the time series from the node. The Distributed Series Reader pulls data in a streaming manner from each Local Series Reader and merges it into fully ordered batches. The batches are returned to the upper level like they were generated by a normal reader. Alignment is applied when specified, to merge data batches of different time series from the Distributed Series Readers, as in Section 5.4.

Distributed Aggregators work similarly. They build a Local Aggregator on each associated node, retrieve aggregations from each node, and combine the results. If the aggregation is not mergeable (like percentiles), the coordinator (the node that receives the query) will use a Distributed Series Reader instead and compute the aggregation from the raw data. The MEAN aggregation is not directly mergeable. Thereby, instead of transferring the average value, the sum and count are sent, making the computation mergeable.

With these basic operators having their distributed implementations, queries can be executed without even noticing the existence of a cluster rather than a stand-alone machine. The current

implementation supports all queries that are built on top of basic operators, and are highly compatible with stand-alone optimizations.

To overcome the difficulty of long query latency of non-parallel operators, we further accelerate the distributed time series queries with parallel operators. (a) We introduce the parallel data scan and aggregation operators based on the independent data partitions of IoTDB, which significantly reduces IO time. Such operators are able to operate on different TsFiles at the same time. When querying on TsFiles that have overlapped time ranges, irrelevant files and data blocks can be filtered, and aggregation results are acquired without serial computation. The improvement of parallelism reduces the amount of IO and data deserialization time significantly. (b) We devise the delicate push-down rules that generate parallel distributed time series query plans, and increase parallelism of operators. The rules include predicate-push-down, aggregation-push-down, and paging-push-down. The associated operators are pushed-down to maximize the parallelism, while keeping the whole semantic correct. Optimized physical query plans are generated with the above rules, guaranteeing that operators can be executed fully in parallel. The experiments in Figure 24 of Section 8.4.1 show that IoTDB with parallel operators significantly improves the performance and outperforms KairosDB.

6.4 Technique: DYNAMIC Read Consistency

While the aforesaid NB-Raft increases parallelism, the "Early Return" may lead to consistency issues among replicas. With write consistency level QUORUM, i.e., persistence in quorum replicas will be regarded as a successful write for efficiency, we may need to retrieve the quorum replicas for the correct results, a.k.a. read consistency level QUORUM.

Note that update of a sensor reading rarely occurs in practice. That is, replicas are mostly consistent with each other, without two versions of the same data. Intuitively, rather than waiting for the remaining replicas to respond, we propose a DYNAMIC read consistency level [64], returning results based on a part of the replicas responded thus far. The number of replicas to retrieve could be dynamically determined referring to the confidence of obtaining the correct results.

7 TsFile Synchronization

In Section 6, how reads and writes are performed within a cluster is introduced. For massive use cases, multiple clusters will be established, for example, one for each province. The integration among clusters is highly demanded, e.g., for extensive data analysis. To support such data integration, the data media, namely TsFiles, are often synchronized from one cluster to another. We show how the synchronization process can be optimized in the chunk level.

Recall that in Section 1.2, an advantage of our solution is to have the same TsFile used in end devices, edge servers, and cloud nodes, without ETL. As the typical scenarios illustrated in Figure 3, TsFile may be transferred from end devices to the IoTDB engine in edge servers, and then synchronized to the IoTDB cluster for cloud computing. For the latter case with a large amount of accumulated data, efficient TsFile synchronization is vital. Let us first introduce a natural baseline to illustrate the synchronization process below, and then present our advanced solution by fully utilizing both the structure characteristics of TsFiles and features of synchronizing tasks.

To overcome the difficulty of synchronizing large files with redundant information, we introduce several technical contributions in chunking/synchronization, in addition to the engineering effort. (a) In Section 7.3, to perform fast synchronization with limited bandwidth and large files, the technical contribution is to explore the correlation of data and enable batch processing, leading to more compacted TsFiles. (b) In Section 7.4, given the varying loads in different nodes, the technical contribution is to select proper synchronization relays based on the flow statistics. (c) In Section 7.5, noting that the IoT workload may vary from time to time, the technical

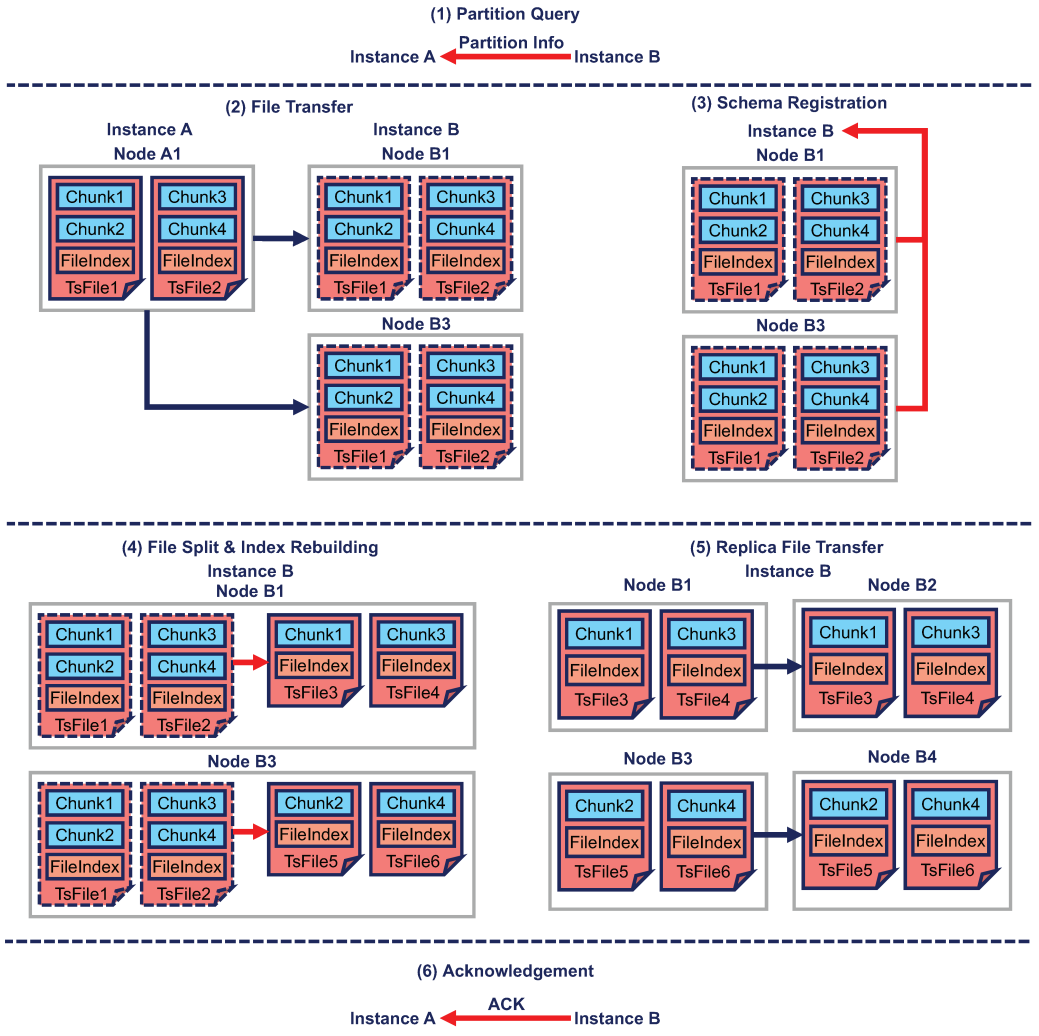


Fig. 13. A baseline of TsFile synchronization with natural file-based transfer. The dark blue arrows denote data transfer through the network, while the red ones are the other flows.

contribution is a highly adaptive synchronization mechanism with adjustable synchronization size. The experiments in Figure 25 of Section 8.5.1 show that the chunk-based synchronization improves significantly not only the synchronization time but also the query latency, thanks to the better compacted chunks in TsFiles during synchronization.

7.1 Baseline: File-based Transfer

Let Instance A denote the source IoTDB having TsFiles to synchronize. For instance, in Figure 13, TsFile1 and TsFile2 in Node A1 of Instance A are about to synchronize to another Instance B of the IoTDB cluster. In the IoTDB cluster, the data are partitioned and replicated as introduced in Section 6.1. For example, Instance B in Figure 13 has two partitions of replica sets, each of which includes two replicas, i.e., Nodes B1 and B2 in Replica Set 1, Nodes B3 and B4 in Replica Set 2. Different parts (chunks) of a file may be distributed and replicated in various nodes in the

cluster, referring to the Partition Table introduced in Figure 12. For instance, Chunk1 of TsFile1 is partitioned to Replica Set 1 and replicated in Nodes B1 and B2, whereas Chunk2 belongs to Nodes B3 and B4 of Replica Set 2.

A natural idea is thus to transfer the TsFile to all the corresponding replica sets, which should carry some parts (chunks) of the data. The overall process of this baseline approach is illustrated in Figure 13 and explained as follows. (1) The Partition Query fetches the destination replica sets for the data of synchronizing TsFiles, from the partition table of Instance B as aforesaid. (2) By knowing the destinations, the files (TsFile1 and TsFile2) are transferred from the source Node A1 to one replica node of each replica set (Node B1 and B3). (3) The received files need to register the schema of stored series in the target Instance B, namely Schema Registration. (4) Each received file is then split to extract the chunks that belong to the corresponding replica set. For instance, a new TsFile3 is created by extracting Chunk1 from TsFile1, and similarly, TsFile4 is generated from TsFile2. The corresponding FileIndex needs to be rebuilt for the cut data. (5) The newly created files are replicated in the other nodes of the replica set, i.e., TsFile3 and TsFile4 are replicated in Node B2. (6) An acknowledgment is sent to the source (Instance A).

This baseline approach overlooks the inner structure of TsFiles introduced in Figure 6 and the correlations among TsFiles for optimizing the synchronization process. First, the entire files are transferred to the target nodes, with redundant data. For example, Chunk2 and Chunk4 in Figure 13 do not belong to Replica Set 1, but are transferred to Node B1 together with TsFile1 and TsFile2 as a whole in step (2). Moreover, different files may store different data parts of the same series, e.g., Chunk1 and Chunk3 belonging to the same series. The corresponding series schema only needs to be registered once in Instance B, rather than repeatedly for each file (TsFile1 and TsFile2) in step (3). These shortcomings motivate us to design an advanced TsFile synchronization scheme below.

Note that chunks with the same name are identical across nodes, as well as for TsFiles. The TsFiles with dashed borders are in temporary storage. Recall that Chunk1 of TsFile1 is partitioned to Replica Set 1 and replicated in Nodes B1 and B2. As illustrated in Figure 13, Chunk1 of TsFile1 from Node A1 is transferred to TsFile3 of Node B1 in step (4) and TsFile3 of Node B2 in step (5). Likewise, Chunk2 belongs to Nodes B3 and B4 of Replica Set 2. As illustrated in Figure 13, Chunk2 of TsFile1 from Node A1 is similarly transferred to TsFile5 of Node B3 in step (4) and TsFile5 of Node B4 in step (5). It is true that a natural idea is to transfer the TsFile to all replica sets, and then filter the necessary part. Thereby, TsFile1's Chunks 1 and 2 appear in both replica sets, as temporary storage denoted by dashed line, before being processed in step (4). After file split, only Chunk 1 is preserved in the new TsFile3 of Node B1, while Chunk 2 appears only in the new TsFile5 of Node B3.

7.2 Design: Chunk-based Transfer

To improve the file synchronization process, we propose to explore the components inside TsFiles, and their relationships. Note that in Section 4.1, all the data in a chunk belong to the same time series. It thus serves as the unit of partition and replication in an IoTDB cluster. Intuitively, rather than transferring the entire TsFile, we may send only the precise chunks to the corresponding nodes, to avoid redundant data. Moreover, different chunks of the same series can be reassembled (with compression) for more efficient data transfer and only need to register the schema once.

Figure 14 presents the overall procedure of the chunk-based TsFile synchronization scheme. The detailed steps are explained as follows.

(1) *Partition Query.* Similar to the file-based transfer in Figure 13, the Partition Query fetches the destination of replica sets in Instance B for the TsFiles in Instance A. It determines the nodes where each chunk (e.g., in TsFile1 and TsFile2) should be sent.

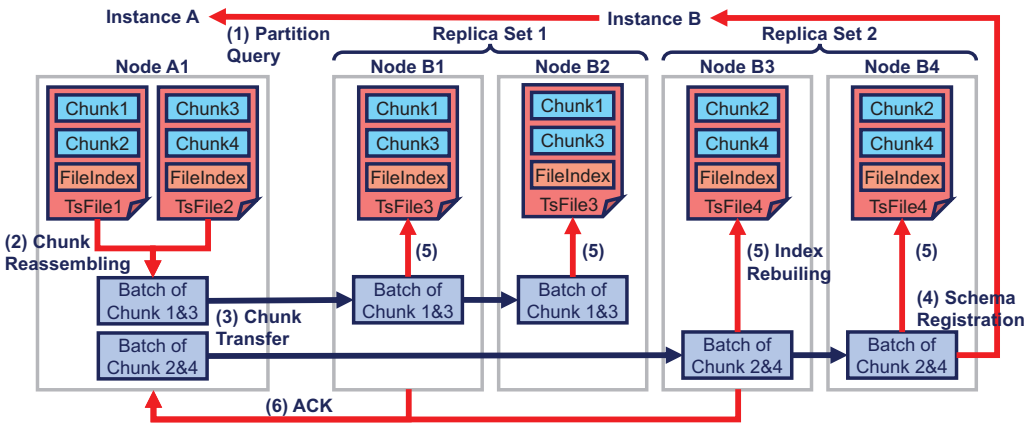


Fig. 14. An overview of TsFile synchronization with advanced chunk-based transfer. The dark blue arrows denote data transfer through the network, while the red ones are the other flows.

(2) *Chunk Reassembling*. The chunks of the same series from different TsFiles are then reassembled together as a batch, e.g., the Batch of Chunk 1&3 in Figure 14. Recall that the FileIndex has to be rebuilt from chunks in the destination in Step (4) of Figure 13. In other words, the FileIndex of the old TsFile does not need to be batched and transferred. Moreover, by fully exploring the similarity among chunks, the chunk batch can be further compressed and thus more efficiently transferred, as presented in Section 7.3 below.

(3) *Chunk Transfer*. The chunks of the same series in a batch have the same destination referring to the aforesaid Partition Table in target Instance B. For example, in Figure 14, The batch of Chunk 1&3 is sent to Replica Set 1, and the batch of Chunk 2&4 is sent to Replica Set 2. For the same replica set, one of its nodes is selected as the relay, e.g., Node B1 of Replica Set 1, which is responsible for forwarding the received chunk batches to the other replicas in the set. The selection of a proper relay will be introduced in Section 7.4.

(4) *Schema Registration*. Recall that each chunk records its schema information as Chunk Metadata, illustrated in Figure 6 in Section 4.1. On the arrival of a chunk batch, the schema of the same time series is read and only needs to register once in the receiver (Instance B). This reduces the number of registrations from once-per-file for each series in Figure 13 to once-per-batch of a series in Figure 14, significantly decreasing the system load caused by checking the existence of time series. In practice, the total number of time series reaches 200 million, which makes the time of redundant registration for different files significant.

(5) *Index Rebuilding*. For each batch of received chunks, a new TsFile is created by rebuilding the FileIndex. Since the chunks of the same series have already been batched together, the generated TsFiles are more compact. For example, Chunk1 and Chunk3 are in the same TsFile3 in Figure 14, which are stored separately by file-based transfer in Figure 13. The more compact storage is obviously more efficient in query processing on the same time series. Note that the more the TsFiles are transferred, the longer the index rebuilding time will be, which may lead to timeouts. Thereby, we discuss the size control for synchronization in Section 7.5.

(6) *Acknowledgment*. When all the new TsFiles are generated on the receiving replica sets, the TsFiles are loaded in each node by a command sent by the source instance. This step makes TsFiles visible to queries. To guarantee consistency, the same consensus protocol introduced in Section 6 is used to replicate the command of loading TsFiles in each node. Finally, the TsFile synchronization

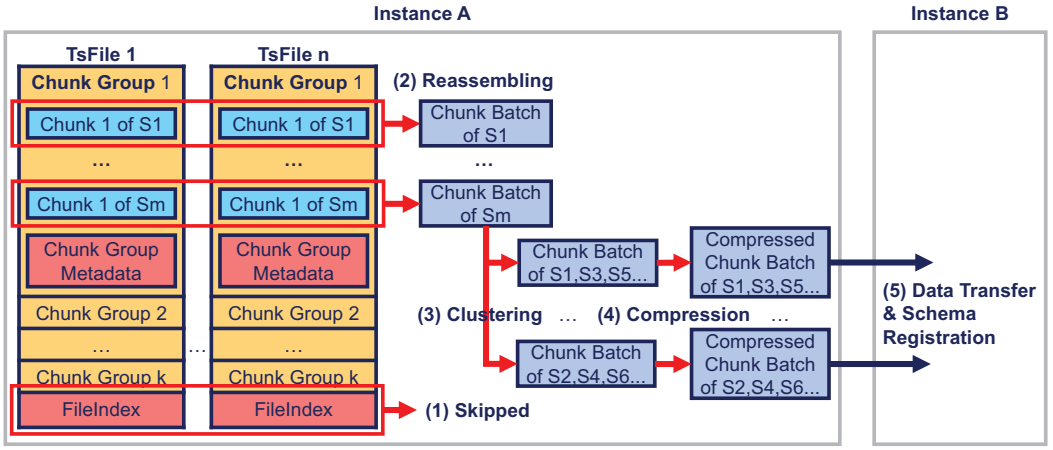


Fig. 15. The process of generating chunk batches for more efficient data transfer and schema registration in Figure 14(2)-(4). The dark blue arrows denote data transfer through the network, while the red ones are the other flows.

is accomplished by an acknowledgment. In Figure 14, the sources of step (6) are Nodes B1 and B3, namely the direct receivers of Node A1.

7.3 Technique: Batch and Compress Chunks

To achieve more concise data transfer and single schema registration for each series, batch of chunks instead of individual chunks are transferred, as in Figure 14. When chunks are batched, the similarity or correlation of data inside them can be utilized to perform compression, which may reduce the message size to transfer chunks and thus the transmission time. Also, batching chunks avoids a series being transferred in multiple messages, removing redundant schema registration.

Figure 15 presents the process of generating chunk batches for more efficient data transfer and schema registration. (1) The FileIndex is skipped, rather than transferring the entire TsFile with the useless index (needs rebuilding) in Figure 13. (2) Chunks of the same time series in different files are then batched. (3) Moreover, chunk batches of different series are further clustered for more similar data. (4) The clustered chunk batches are then compressed and sent to the target Instance B, i.e., step (5) for single schema registration.

7.3.1 Reassembling Chunks of the Same Series. Recall that chunks of the same time series are scattered in different files. As shown in Figure 15, each file (TsFile 1, ..., TsFile n) contains only one chunk of series S1, which may be written at different times. Instead of processing TsFiles in a file-by-file manner, we propose to reassemble chunks of the same series from different TsFiles within a batch, and process series-by-series.

Note that chunks in TsFiles are sorted according to their series names, such as S1, S2, ..., and Sm as presented in Figure 15. Therefore, we can easily read the chunks of the same time series from different files, without breaking the scan order. In other words, reassembling chunks of the same series is efficient by scanning TsFiles. After reassembling, each series corresponds to one batch, such as the Chunk Batch of S1 in Figure 15 consisting of Chunk 1, ..., Chunk n of S1 from TsFile 1, ..., TsFile n, respectively.

The benefits of reassembling chunks of the same series are in three folds. (1) Each series batch has a single schema registration for multiple chunks of the same series, in the target Instance B, given the same series schema. (2) As discussed in Section 4.3, the data of the same series are often

stable and similar, excellent for compression. Thereby, chunk batches of the same series can be further compressed and more efficiently transferred. (3) Finally, by batching the chunks together, it is not surprising that future queries on the same series can be more efficiently processed.

7.3.2 Clustering Chunk Batches of Different Series. To push it one step further, we may further cluster the time series with high similarity, for even better compression. For example, in Figure 15, the Chunk Batches of S1, S3, S5, ...are grouped together in compression. By exploring the similar time series, more redundant information can be detected and removed by the compression algorithm, thus reducing the bandwidth consumption during transfer.

The cost of clustering may be a concern. Note that the clustering is performed at the series level, rather than chunks or specific data points. Thereby, we can utilize the schema information stored in the Chunk Metadata of chunks as the features of time series for clustering. First, the name of a time series provides rich semantics, e.g., "speed", "temperature", and "voltage", which can be used to group the time series. Together with other information important to data compression [69], such as data type, min/max/avg values, and so on, it forms the feature vector of a time series. Efficient clustering algorithms like K-means are then conducted. Thereby, the clustering on series with features from metadata does not incur much extra cost of data transfer.

7.4 Technique: Select Relay for Replica Set

There are multiple nodes in a replica set in the target Instance B of the IoTDB cluster, as illustrated in Figures 13 and 14. Note that the network bandwidth between the source and target instances is often not as large as that among different nodes in a cluster. Thereby, rather than transferring the data directly from the source Instance A to all the replica nodes of the target Instance B, a replay node is selected to transfer the data to other replicas. For instance, Node B-1 is the relay of Replica Set 1 in both Figures 13 and 14. The selection of a very busy relay from multiple nodes in a replica set may affect the efficiency of data transfer. Since loads of nodes change from time to time in the cluster, the relay selection should be dynamic rather than static. In the following, we present a dynamic relay selection scheme by monitoring the throughput of a node.

Specifically, the throughput of each transfer (initially MAX) is recorded for a node. The relay selection is periodically triggered, in every $t_{\text{selection}}$ time, e.g., $t_{\text{selection}} = 100$ in Figure 16. The probability of a node being chosen as the replay is proportional to its throughput.

Sometimes, performance-interfering events like garbage collection or load spikes may occasionally reduce the performance of a node, temporarily resulting in very low throughput. It may cause the node not to be chosen for a long time, even if it has already recovered from the performance-interfering events. The reason is that without being selected, its throughput remains low in the last record. To increase the probability of selection for the node, a time amplifier is introduced. That is, in each selection, if a node is not selected as a relay, its throughput is multiplied by an amplifier as the weight in selection probability calculation. The weight of a node i is thus given by

$$\text{weight}[i] = \text{throughput}[i] * 2^{\frac{t_{\text{past}}[i]}{t_{\text{selection}}}},$$

where $t_{\text{past}}[i]$ is the elapsed time since the last time of node i being selected. Based on the weights, the probability of node i being chosen as the relay is calculated as

$$\text{probability}[i] = \frac{\text{weight}[i]}{\sum_j \text{weight}[j]}.$$

With the time amplifier, if a node is not chosen as the relay, its probability of being chosen will keep increasing with $t_{\text{past}}[i]$ elapse till it is chosen. Once the node is chosen as the relay again, its throughput can be updated in the transfer.

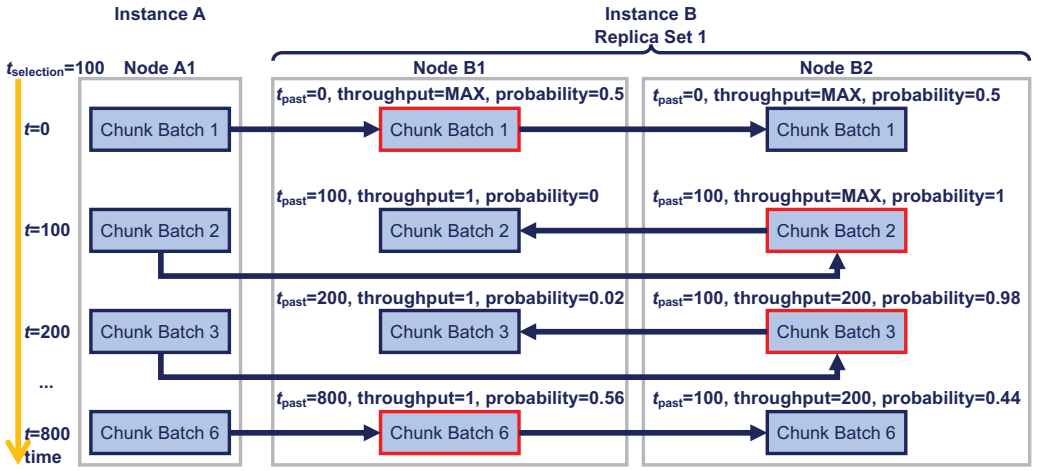


Fig. 16. Throughput-based relay selection for efficient transfer in Figure 14(3).

With the same t_{past}, the smaller the selection time $t_{\text{selection}}$ is, the larger the weight will be. It means that the node i with a long time not being chosen is more likely to be chosen now. However, a lower $t_{\text{selection}}$ makes the selection on nodes more frequent. Consequently, the node affected by garbage collection or load spikes may not be recovered yet. Thereby, a moderately large selection time $t_{\text{selection}}$ is preferred.

Example 7.1. Figure 16 presents an example of the relay selection process. Consider a selection time $t_{\text{selection}} = 100$. Instance A wants to send chunk batches to Replica Set 1 of two nodes, Node B1 and Node B2, as in Figure 14. At timestamp 0, in Figure 16, both nodes have not been chosen yet. Their throughput is initialized as MAX; thus, they have the same probability of being chosen (0.5). As shown, Node B1 is chosen randomly as the relay for Chunk Batch 1 (in red). Suppose that a major garbage collection happens on Node B1 during receiving Chunk Batch 1, resulting in a very low throughput 1. At timestamp 100 in Figure 16, since Node B2 has not been chosen, it is selected this time and performs transfer normally. At timestamp 200, owing to the aforesaid low throughput, the probability of Node B1 is still small

$$\frac{1 * 2^{\frac{200}{100}}}{1 * 2^{\frac{200}{100}} + 200} = 0.02.$$

Therefore, for the following several batches, batches are all sent to Node B2. After some time, at timestamp 700 in Figure 16, the time amplifier boosts the probability of Node B1 to 0.56, given the same throughput.

$$\frac{1 * 2^{\frac{800}{100}}}{1 * 2^{\frac{800}{100}} + 200} = 0.56$$

It is high enough to be selected as the relay of Chunk Batch 6, which has already been recovered from the aforesaid garbage collection.

7.5 Technique: Dynamically Adjust Synchronization Size

As shown in Figure 14, we have multiple TsFiles processed in one synchronization for chunk reassembling. When the source Instance A has several hundred GBs of TsFiles, we cannot send them all in a single synchronization. The reason is that such a huge job will incur timeout. When all

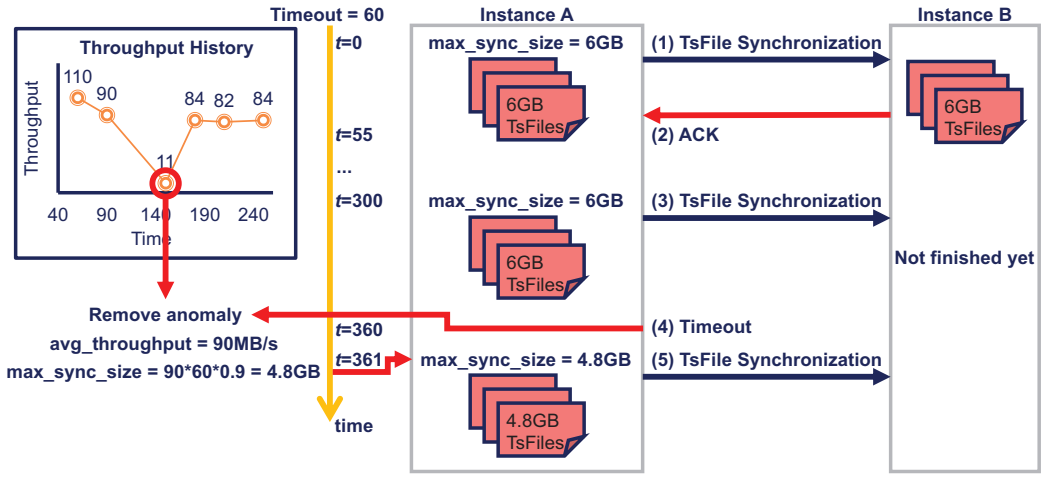


Fig. 17. Throughput-based synchronization size control during reassembling in Figure 14(2) to avoid timeout.

chunks of a synchronization are sent to their destination nodes, the schema of the corresponding series will be registered, and the FileIndex needs to be rebuilt for the received chunks in each node, as indicated in Figure 14 (4) and (5). A very long time of chunk transfer, schema registration, and index building will incur timeouts and trigger a retry. Note that we cannot arbitrarily change the timeout setting, since it is related to other functionalities like node failure detection. Thereby, the total size of TsFiles in a synchronization should be controlled to avoid unnecessary retries, namely the maximum synchronization size (max_sync_size).

To determine a proper synchronization size, we again refer to the throughput in the history. In Figure 17, the history records the throughput of recent synchronization tasks that are successfully accomplished. When a timeout is detected, the synchronization size is dynamically adjusted according to the previously recorded throughput, e.g., in the last five synchronization tasks,

$$\text{max_sync_size} = \text{avg_throughput} * \text{timeout} * 0.9.$$

For instance, the system timeout is set to 60 in Figure 17. Intuitively, without reaching the timeout, the synchronization size should be large enough to enable the reassembling of multiple TsFiles in Figure 14(2). The synchronization size is determined by the average throughput in the history multiplied by the maximum duration of the timeout. Considering that throughput fluctuates, we further multiply it by a safety factor of 0.9.

Again, the synchronization throughput is also affected by occasional events like garbage collection. For instance, the throughput of the synchronization around timestamp 140 in Figure 17 is very low, only 11. Obviously, the synchronization size of most other tasks without such events should not be affected. To reduce the disturbance by such anomalies, we need to first remove such abnormal events before computing the average throughput.

Example 7.2. Figure 17 illustrates an example of dynamically adjusting the synchronization size referring to the throughput history. The system timeout is set to 60, i.e., synchronization time longer than 60 will be considered as timeout. The initial synchronization size is limited to 6GB, reassembling no more than 6GB TsFiles. At time $t = 0$, Instance A sends 6GB TsFiles to Instance B. Once steps (1)-(6) in Figure 14 are successfully executed, an acknowledgment is returned at time $t = 55$ (before reaching timeout). Its throughput of 110MB/s is then added to the history in Figure 17. After several synchronizations, at time $t = 300$, Instance A again sends 6GB TsFiles to Instance B.

Table 3. Summary of Datasets from Different Domains

Dataset	# time series	# data points	total size
TY-vehicle	28,781	1,511,347,655	56G
ZC-train	93,284	46,708,284,999	248G
ZY-machine	111,027	17,580,821,079	735G
TSBS	300,000	92,897,280,000	70.8G

However, the recent throughput has decreased to around 80MB/s somehow, e.g., owing to more background queries. Consequently, at time $t = 360$, Instance A reaches timeout before finishing this synchronization. Therefore, Instance A calculates the average throughput from its history (without anomalies) as 90MB/s. In order to finish synchronization without incurring timeout, the size should be smaller than the new max_sync_size

$$90\text{MB/s} * 60\text{s} * 0.9 = 4.8\text{GB}.$$

Recall that we have timeout=60 and safety factor 0.9 with the consideration of performance fluctuation. From time $t = 361$, Instance A adjusts the max_sync_size=4.8GB for later synchronizations.

8 Experiments

We compare TsFile and IoTDB separately to the state-of-the-art file formats and time series databases that are widely used in the industry.

8.1 Experimental Settings

8.1.1 Hardware. For the stand-alone version, we conduct the experiments on an 8-core Intel(R) Core(TM) i7-10700 CPU @ 2.90GHz machine with 32GB memory and 10T HDD, running 64-bit Ubuntu Server 16.04.7 LTS. We also use another PC as the client in the database experiment. The machines are connected to a 10GB network.

For the distributed version, we use machines from Alibaba Cloud with Intel Xeon (Cascade Lake) Platinum 8269 2.5 GHz/3.2 GHz CPU, 256GB memory and PL1 ESSD.

8.1.2 Baselines. We compare TsFile v0.13.0 with two columnar file formats: Parquet v2.9.0 [51] and ORC v1.5.4 [29, 49]. For time series database competitors, we compare IoTDB v0.13.0 in three typical categories: time series native InfluxDB v1.8.10 [30], KairosDB v1.2.2 [38] on NoSQL database Cassandra v3.11.2 [41], and TimescaleDB v1.7.5 [65] on RDBMS PostgreSQL v12.7 [55].

8.1.3 Datasets. We employ three real datasets collected by our industrial partners and a time series benchmark, as listed in Table 3.

TY-vehicle. The TY-vehicle data are collected from excavators and delivery vehicles of Tianyuan Technology, an engineering machinery company. The sensors monitor vehicle speed, fuel gauge readings, acceleration, and 1,033 other measurements. The data set is very sparse as the time series has varying frequencies. By aligning time series as in Figure 5, only 4.79% data are not null.

ZC-train. The ZC-train data are collected by monitoring trains in Shanghai subway, manufactured by the CRRC Corporation Limited. The 4,705 measurements include temperature, speed, load, and so on. Because data are collected and uploaded by trains using a batched manner, the dataset is relatively dense, and 72.8% of it is not null.

ZY-machine. The ZY-machine dataset consists of the production logs of machines in China Tobacco. 117 indices like water content, batch number, and rod number are recorded to monitor the

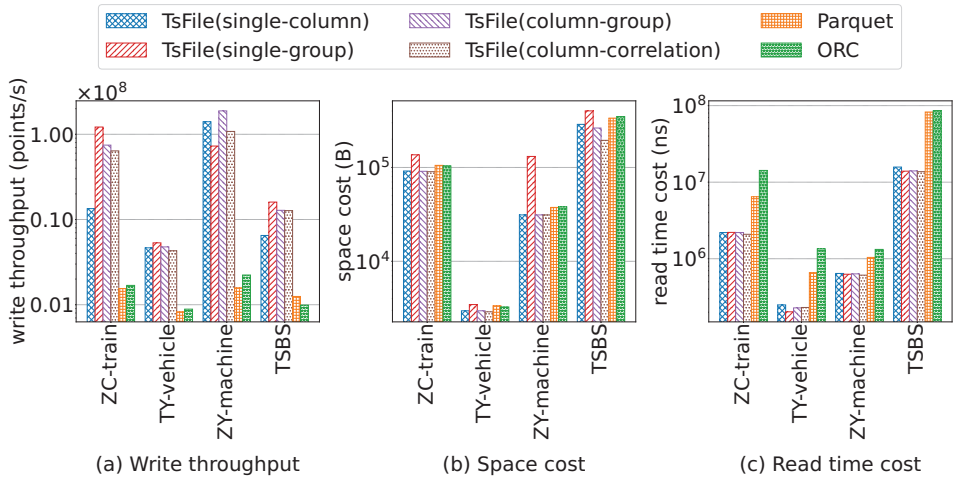


Fig. 18. File evaluation with various time series alignment schemes in TsFile.

production. Since different types of machines work concurrently, the dataset is also very sparse, with 88.5% null values.

TSBS. The TSBS dataset is generated by the commonly used Time Series Benchmark Suite [66], consisting of 400 hosts, each managing 9 devices (3,600 devices in total). The devices take samples every 15 seconds without any null values. The time range of the dataset is 40 days, making the total size of the data set 70.8GB.

8.1.4 Queries. We consider four main query types.

Q1 – Time range query

```
select series from device where time > ? and time < ?
```

Q2 – Alignment query

```
select series1, series_2, ... from device
```

Q3 – Aggregation query

```
select count(series) from device
```

Q4 – Downsampling query

```
select count(series) from device group by ([start_time,
end_time), sampling_interval)
```

8.2 File Comparison

We evaluate the performance of TsFile, Parquet, and ORC on write throughput, disk occupation, and read time cost. Figure 18 reports the results over four data sets.

We use a non-aligned schema, {time, deviceId, measurementId, value}. This schema avoids storing nulls in the file. It is storage efficient for IoT scenarios where null values are generated due to inconsistent sampling frequencies or various start points of different sensors and devices. Since Parquet and ORC are not specially designed for IoT scenarios, they store deviceId and measurementId as normal columns. In contrast, TsFile stores them in chunk headers, and thus, duplicated information is significantly reduced.

TsFile uses insertTablet as the writing interface, and Parquet and ORC use their default writing interfaces respectively. All three file formats perform lossless compression and encoding of data.

Parquet and ORC have auto-encoding turned on, which can choose the most appropriate encoding method for each column based on the data type and compression efficiency.

In terms of queries, both TsFile and Parquet support filtering, and only data that meets conditions are returned. ORC will return the entire block of data that may contain query results, which requires further filtering by the upper layer.

8.2.1 Write Performance. The results in Figure 18(a) show that TsFile has a superior write performance over all other files, as its IoT-aware structure design avoids storing redundant information like deviceId and measurementId. TsFile and ORC provide a vectorized write interface suitable for IoT data ingestions which usually stream in as small batches. Such a vectorized interface improves memory locality and results in better performance.

The performances are also affected by data type and encoding. Parquet uses dictionary encoding as the first choice, but if the size of the dictionary and encoded data exceeds the raw data size, it falls back to another encoding like RLE. Consequently, Parquet writes the same data twice from time to time, and thus has a low throughput. ORC uses a dictionary encoding backed by a red-black tree, which is computationally expensive and slows down its performance.

8.2.2 Space Cost. The space cost is reported in Figure 18(b). TsFile has compatible space efficiency compared with other formats. It avoids storing duplicated deviceId and measurementId as data columns but builds finer indices that occupy more space. As shown in the following experiments, this sacrifice brings extraordinary improvement over query time, i.e., worthwhile. The duplicated deviceId and measurementId can be removed by dictionary encoding to some extent, but when the number of unique IDs is large (especially in the TSBS data set), they will still occupy a large space, making Parquet and ORC less feasible.

8.2.3 Read Performance. Remarkably, the read performance of TsFile is clearly better than all the competitors in all tests, as illustrated in Figure 18(c). The results are not surprising, given the indices and statistics stored in TsFile. Such a significantly lower read time cost verifies again the necessity of developing a time series native file format. ORC is slower due to its absence of supporting filters, and the application-level filtering is less efficient.

8.2.4 Single-Column Compression. For storage format, we experimentally compare our TsFile with storage and compression solutions TVStore [2], MOST [71] and BUFF [44], in Figure 19. Since TVStore [2] and MOST [71] are lossy compression, in addition to space cost, we further compare the RMSE loss and accuracy between the decompressed data and the original data. As shown in Figure 19, TsFile has significantly better space cost than BUFF, which is also lossless compression, having RMSE loss 0 and accuracy 1. While the space of TsFile is comparable to either TVStore or MOST, the latter is lossless, with high RMSE loss and low accuracy of decompressed data.

8.2.5 Multi-Column Compression. We compare four ways of organizing time series: (a) *single-column*, each time series has its own time column; (b) *single-group*, all time series are aligned sharing the same time column; (c) *column-group*, time series are aligned in several groups according to their similar timestamps; and (d) *column-correlation*, time series are aligned in groups with value correlations. The experimental results in Figure 18 show that the original single-column method performs well in dataset ZY-machine with many nulls, where the single-group method fails at the cost of significantly higher space cost. The column-group approach by selecting proper groups of time series for alignment can avoid the bad cases of both methods. All proposed methods show similar read performance, significantly faster than other baselines.

The experimental results also reveal that for datasets with strong time series correlation like TSBS, the correlation compression (i.e., the column-correlation method) can be used to save about

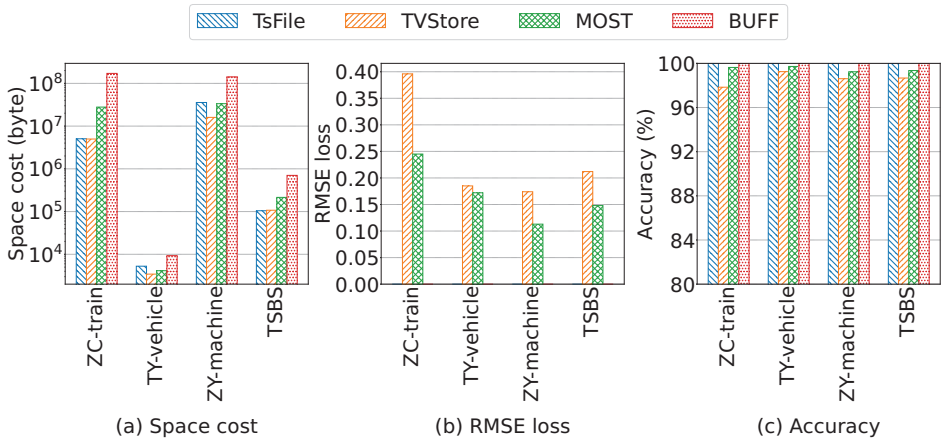


Fig. 19. Comparison with storage and compression solutions over different datasets.

30% space. There is almost no extra overhead in write and read, at the same time, owing to the saved IO cost of smaller space. However, when data correlation is weak in other datasets, the space cannot be saved but the write throughput is slightly downgraded due to the correlation calculation.

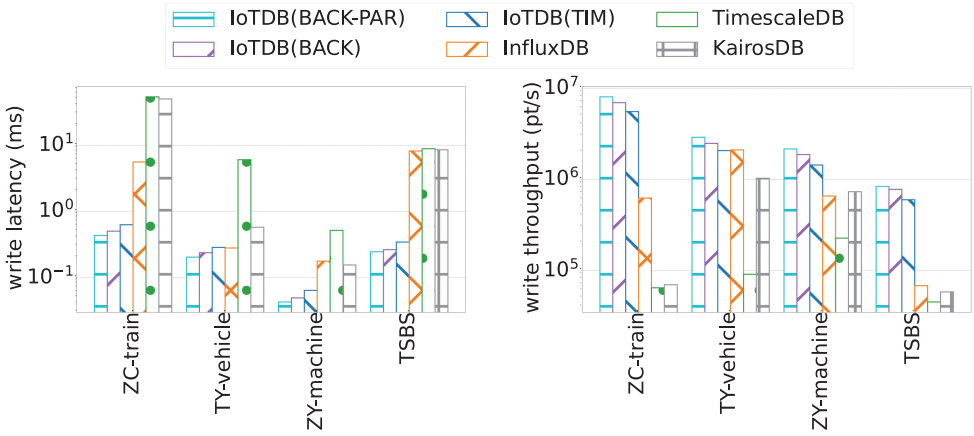
8.3 Database Comparison

We compare IoTDB with InfluxDB, TimescaleDB, and KairosDB on write and query performances. When writing data, IoTDB uses the Java native interface insertTablet(), and the JVM memory is set to 20G. For InfluxDB, time series are stored as one measurement with one tag. The retention policy defaults with an infinite duration, resulting in a one-week shard duration [34]. The cache-max-memory-size is set to 0 to avoid writing failure. Influxdb-java client library [32] is used. For TimescaleDB, we create one regular table that includes a time, a series ID, and the metric columns. Then we create a hypertable which is partitioned by time. The time interval used for time partitioning in TimescaleDB is one week. We use the default configuration of KairosDB and its backend store Cassandra.

8.3.1 Write Performance. Figures 20(a) and 20(b) present the write latency and throughput, respectively. In general, IoTDB shows better performance in almost all tests, higher write throughput as well as lower write latency. The improvement by IoTDB is limited compared to InfluxDB, in the TY-vehicle data in Figure 20. The reason is that this dataset contains a large proportion of strings, which is not specialized by either system, and thus the systems do not show much difference. As both IoTDB and InfluxDB use time-series-oriented storage implementation, they are better tuned for time-series ingestion than PostgreSQL of TimescaleDB or Cassandra of KairosDB.

8.3.2 Sorting Evaluation. For the backward sort technique for delayed arrivals in Section 5.2, we conduct an experiment in Figure 20. We compare our backward sort solution IoTDB(BACK) and its parallel optimization IoTDB(BACK-PAR) with a baseline of TIM sort [53] IoTDB(TIM) and other databases across different datasets. The experimental results in Figure 20 show that the backward solution IoTDB(BACK) can reach a higher write throughput and lower latency compared with the baseline solution IoTDB(TIM). The parallel optimization IoTDB(BACK-PAR) further improves the performance by 10-20%.

8.3.3 Query Processing. The number of data points and time series affects the query performance. Therefore, we use the benchmark to generate 100 time series, including 10 devices with 10



(a) Write latency (amortized by batch size) (b) Write throughput

Fig. 20. Database write performance with various sorting methods in IoTDB.

sensors. In particular, each time series contains about 3 years of data, for a total of 10 billion data points in the entire dataset. The data type is Double, and the time interval between adjacent data points is 1 second. Figure 21 reports the results of the range query, alignment query, aggregation query, and downsampling query. The experiments vary query time ranges, the number of time series (sensors), and sampling intervals.

For the time range query in Figure 21(a), the query latency of IoTDB is higher when the queried time range is small. The reason is that the IO unit of IoTDB is a chunk. The default size of a chunk is equal to or larger than 64K. Therefore, although the queried time range is small, IoTDB still needs to read a chunk and unpack a data page, thus reading more data than the query wanted. Although more data are read, the query latency is still acceptable. For the same reason, the aggregation query of IoTDB shows higher time cost for a time range of 1h in Figure 21(c).

The improvement of IoTDB is not significant when the number of queried sensors is small in Figure 21(b). The reason is that the alignment query needs to start new threads for each series, which is more meaningful when the number of queried series reaches a certain amount.

Thanks to the multiple-level statistics of IoTDB, including page-level, chunk-level, and file-level, down-sampling queries can use higher-level statistics when the sampling interval becomes larger. Consequently, the latency is reduced as it does not need to read disk chunks or decode pages, which is shown in Figure 21(d).

The results demonstrate that IoTDB shows better performance when the queried data are large in scale. Remarkably, the superiority of IoTDB is especially significant in aggregation over large-scale data, where the pre-computed statistics in TsFile (introduced in Section 4.1) perform.

8.3.4 Machine Learning Queries. In addition to AR models, we further evaluate queries on ARIMA models to demonstrate that IoTDB can support various types of ML query optimization. The experimental result in Figure 22 shows that with less than 10% higher write latency and almost no additional space cost, the query latency of advanced LSM-AR and LSM-ARIMA is reduced by about one order of magnitude, compared to the original AR and ARIMA queries, respectively. The space cost in Figure 22(b) around $0.9 * 10^6$ points show a slower growth because data points in that range have more duplicates, leading to a better compression.

8.3.5 Edge Database Comparison. For database engine, we experimentally compare our IoTDB with edge database solutions SQLite [25] and DuckDB [57] in Figure 23. Since the source code of

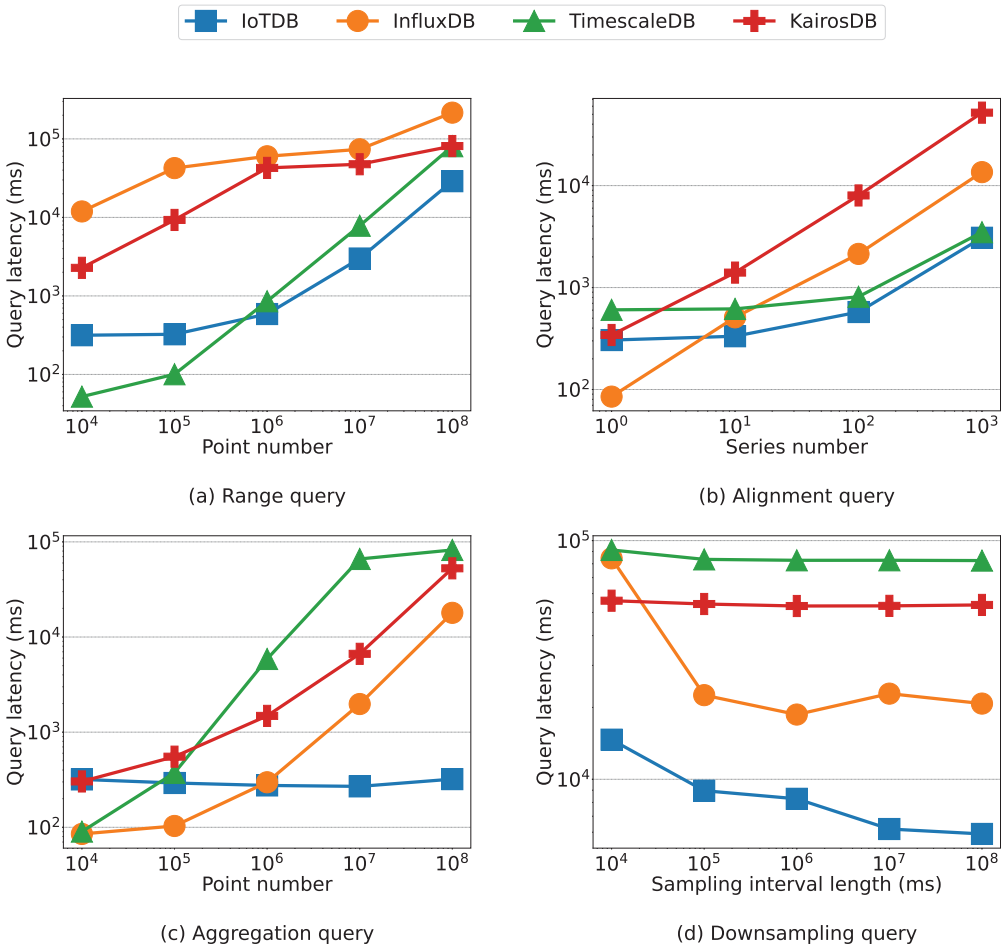


Fig. 21. Query processing over various databases.

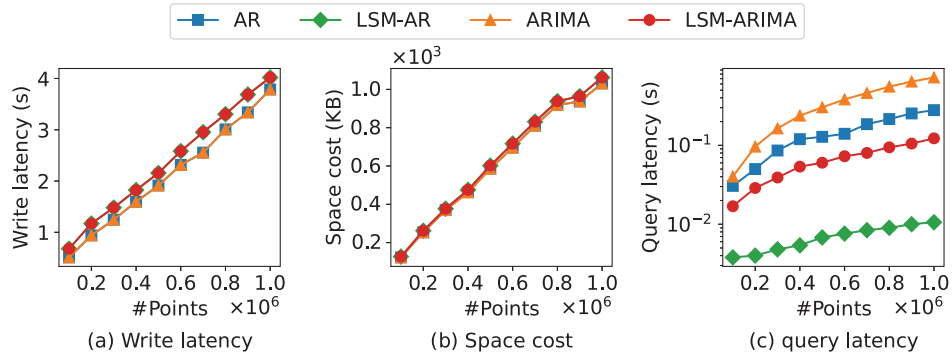


Fig. 22. In-database machine learning query evaluation with AR and ARIMA models.

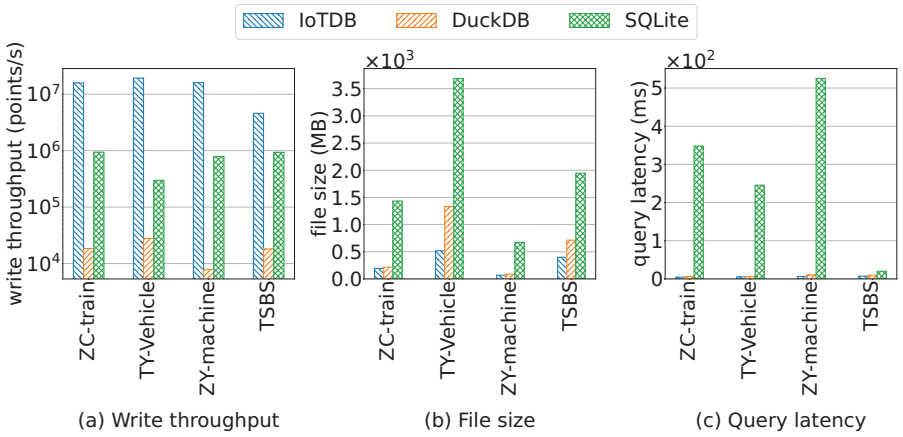


Fig. 23. Comparison with edge database solutions over different datasets.

VergeDB [50] and TransEdge [60] is not available, we cannot compare with them directly. Instead, we choose two other popular edge databases, i.e., SQLite [25] and DuckDB [57], for comparison. The results in Figure 23 demonstrate that IoTDB is significantly faster in write performance. Since the other two databases do not treat data compression as the first-class citizen, their space cost is no better than IoTDB, which is vital for edge devices that with limited storage. DuckDB is more optimized for read than write, with query latency close to IoTDB. SQLite query is faster on dataset TSBS, which consists of many small tables, leading to low data processing cost.

8.4 Distributed Solution Evaluation

The evaluation of the distributed solution is in two aspects, (1) how the compacted TsFile improves data synchronization between nodes in a cluster, and (2) how the IoTDB cluster preforms compared to the existing solution.

8.4.1 IoTDB Clusters. For the IoTDB cluster, we compare it with the KairosDB cluster, which is open source. We again use the benchmark to write 10 million time series with 0.1 million devices and 100 sensors in each device. The data type is still Double, and the time interval between adjacent data points is 1 second. The number of benchmark nodes is the same as database nodes. Each benchmark node is associated with 10 clients, i.e., the total number of clients varying from 50 to 200. Figure 24 reports the write and read performance on different nodes in the cluster, varying from 5 to 20. The replication factor is 3 for both databases. IoTDB shows about an order of magnitude improvement on write throughput compared to KairosDB and much lower write latency.

We compare IoTDB without/with parallel operators, denoted by IoTDB(non-parallel-operators) and IoTDB(parallel-operators). The latter allows time series operators to be executed on more nodes in parallel, for better resource utilization. The results in Figure 24 show that after such optimizations, IoTDB is able to perform better in larger clusters, leading to a 1.5x throughput and 1/10 query latency. However, as the resource contention increases with optimization, write latency is higher but still below 100 ms. Thus, IoTDB still supports high rate ingestion of about 10 Hz time series.

8.5 Advanced TsFile Synchronization

In this experiment, we evaluate the advanced TsFile synchronization with a chunk-based transfer, proposed in Section 7. As in Figures 13 and 14, a standalone IoTDB instance serves as the source

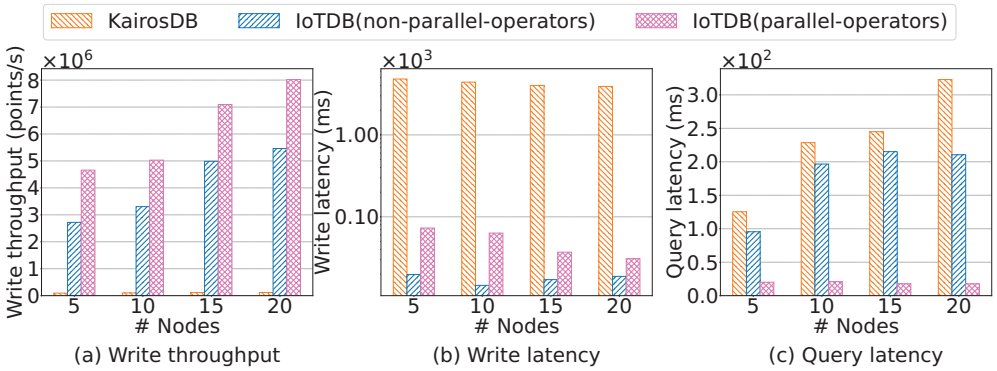


Fig. 24. Distributed solution evaluation with/without operator parallelization in IoTDB.

Instance A and an IoTDB cluster with three replicas as the target Instance B. The source Instance A synchronizes about 800GB of TsFiles data to the target Instance B. This large-scale dataset is generated by the TSBS benchmark aforementioned in Section 8.1.3. In Section 8.5.1, we first present the overall performance of the advanced synchronization with chunk-based transfer in Figure 14, compared to the baseline of file-based transfer in Figure 13. Then, we evaluate the improvement by the individual techniques proposed in Sections 7.3–7.5, respectively.

8.5.1 Overall Evaluation. We first present the overall improvement by applying all optimization techniques, compared with the baseline approach aforementioned in Section 7.1. As considered in Figure 13 and 14, TsFiles are synchronized to multiple replica nodes in the target Instance B of the IoTDB cluster. The experiment in Figure 25 varies the number of replicas. Two measurements are taken in this evaluation, synchronization time and query latency. The synchronization time is the time cost of synchronizing the whole dataset. After the dataset is synchronized, we perform an aggregation query as mentioned in Section 8.1.4 on the target instance and record the query latency. This will verify the benefit of chunk reassembling introduced in Section 7.3.1.

As shown in Figure 25(a), the fine-grained chunk-based TsFile synchronization method shows up to one-order-of-magnitude improvement in time consumption, compared to the baseline with the file-based transfer. The bandwidth saved by skipping index, grouping, and compressing chunks greatly reduces the synchronization time. Moreover, the chunk-based TsFile synchronization also speeds up the query by about 30% in Figure 25(b). The benefit comes from the improved data locality by reassembling chunks of the same series, a by-product during TsFile synchronization as discussed in Section 7.3.1.

8.5.2 Chunk Batch and Compression. To further illustrate the advantage of chunk-based transfer in Section 7.3, we present a more detailed comparison with the file-based synchronization, in Figure 26. First, the experiment varies the number of files, i.e., TsFile 1 to n as indicated in Figure 15(2), determining how many chunks of the same series can be reassembled together. Moreover, the number of series in one file, i.e., series S1 to Sm, affects the result of clustering in Figure 15(3). Finally, the chunk size, i.e., the number of points in a chunk, is related to the performance of compression in Figure 15(4). The product of the above three determines the total data set size.

The time cost of synchronization increases almost linear to each of the parameters, except Figure 26(b). The increase of the chunk-based method is slower by varying the number of series in a TsFile. The reason is that by increasing the number of series, more similar series can be grouped together in a cluster, which leads to even better compression and thus a lower transfer cost. The

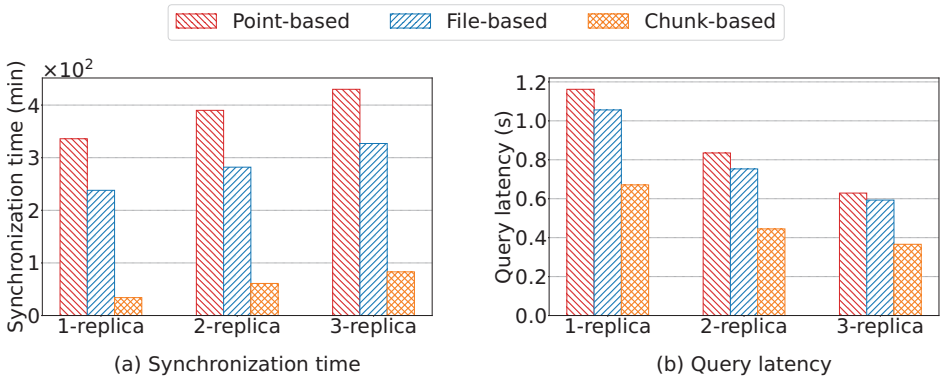


Fig. 25. Comparison of chunk-based TsFile synchronization, file-based transfer baseline, and point-wise insertion baseline.

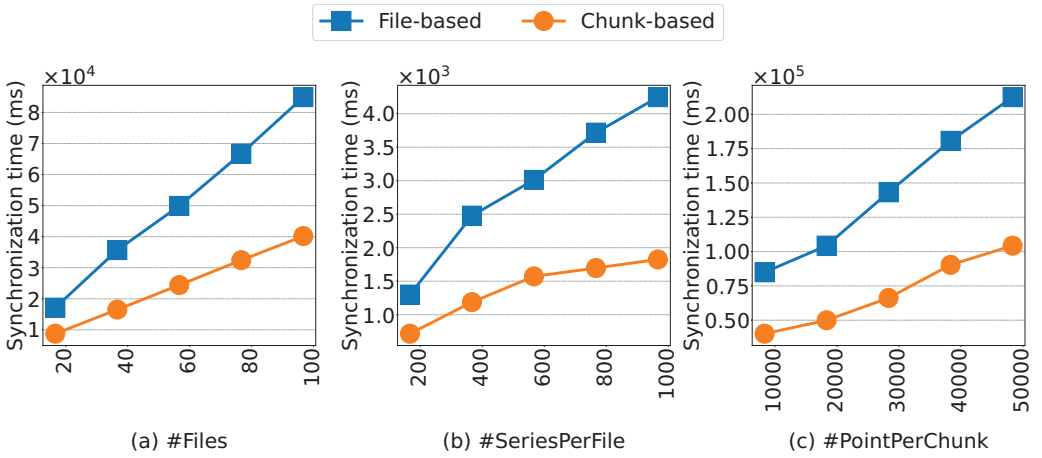


Fig. 26. Improvement of TsFile synchronization by chunk reassembling.

results indicate that with chunk reassembling and clustering as in Figure 15, the synchronization time cost is reduced by about half, a significant improvement.

8.5.3 Relay Selection. As mentioned in Section 7.4, given the slower connection between the source Instance A and the target Instance B, the source instance will select a replica node from the replica set as the replay to propagate chunk batches to the other replicas in the target instance. The relay selection as illustrated in Figure 16 may affect the performance of synchronization, referring to the various loads and network conditions of different replica nodes. To simulate such circumstances, we inject one replica with latency and packet loss ratio obeying the normal distribution of some average. Another replica also has latency and packet loss, with half of the average. The last node has no latency and packet loss injected. We compare the proposed throughput-based relay selection with the simply fixed relay and the randomly selected relay in Figure 27.

Figure 27(a) shows that as the latency increases, selecting a fast replica as the replay becomes more important, and more time is saved. Figure 27(b) also indicates the importance of selecting a proper relay under different packet loss ratios. Packet loss results in retransmission and lower

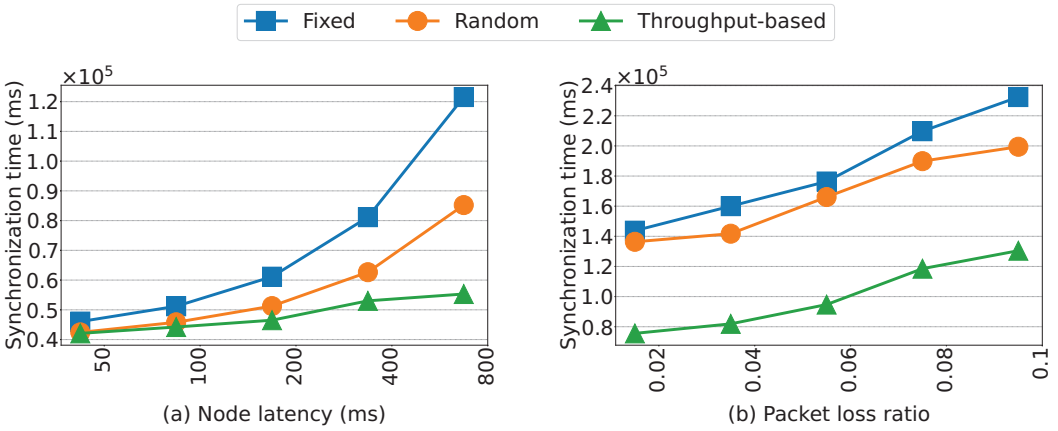


Fig. 27. Evaluation of TsFile synchronization by different relay selection strategies.

throughput, which can be captured by the throughput history introduced in Figure 16 in Section 7.4. The throughput-based method avoids choosing the node with high packet loss as the relay.

8.5.4 Synchronization Size. To avoid timeout, as illustrated in Figure 17 in Section 7.5, the total size of TsFiles in synchronization should be controlled. In this experiment, we demonstrate the effectiveness of dynamically adjusting the maximum synchronization size by comparing it with the manually fixed synchronization sizes.

As illustrated in Figure 28, if the synchronization size is too small, e.g., 200MB, the synchronization time cost is high. The reason is that insufficient TsFiles and series can be reassembled in Figure 14(2). On the other hand, if the synchronization size is too large, such as 1000MB, it may incur timeout and retry, as explained in Figure 17. The overall synchronization time is high as well. As shown, a proper synchronization size like 600MB shows the best performance. While it is difficult for a user to specify manually such a size, the dynamic size adjustment can successfully control the synchronization size via throughput history as in Figure 17, and shows the lowest synchronization time cost.

9 Related Work

In this section, we discuss related work in file storage and time series databases. While some of the features are compared in Tables 1 and 2, respectively, more details are explained below.

9.1 File Storage

The data placement structure affects the data management system performance in a fundamental way [28]. For the requirements of big data analysis, there are many file formats presented, such as Parquet [51], Avro [14], ORCFile [29], and RCFile [28]. They integrate well with the data analysis systems, such as Impala [21], SparkSQL [13], MapReduce [19], Spark [72], and so on. Unfortunately, these big data processing oriented file formats do not fit well the IoT applications. They do not consider the features special in the IoT scenarios as summarized in Section 1.1.

Apache Parquet [51] adopts the nested data model proposed by Dremel [45], which is suitable for the semi-structured data such as XML or JSON documents, but not for time series data with ever-evolving schema in the IoT scenarios. It can be adapted to store deviceId and measurementId as normal columns, resulting in redundant information being stored. In contrast, TsFile stores them in the chunk metadata headers, saving space for indices on seriesID for fast access. As a

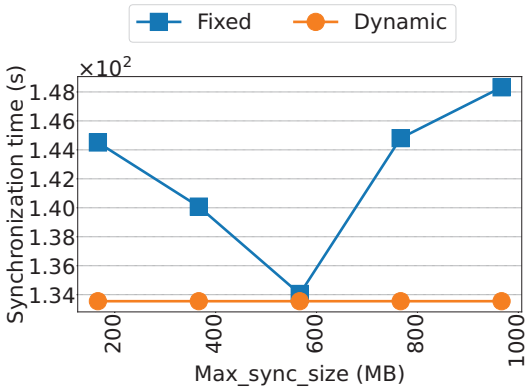


Fig. 28. Comparison of TsFile synchronization with manually fixed and dynamic sizes.

result, TsFile has both lower space consumption and read time cost than Parquet, as illustrated in the experiments in Figure 18. Moreover, Parquet falls back to plain encoding once the dictionary encoding for ids cannot be used (e.g., the dictionary size exceeds the limit of 1MB [73]). The fallback scheme of Parquet can lead to duplicate data writes when there are a large number of distinct ids in IoT applications. Therefore, in Figure 18(a), Parquet has a lower write throughput than TsFile.

Apache ORC [49] is a successor of Hive's RCFile [28], supporting column-based operations such as late materialization. Similar to Parquet, ORC stores IDs as normal columns given the non-aligned schema in Section 8.2, leading to more space occupation, as shown in Figure 18(b). Instead of using the fallback strategy as Parquet, ORC uses the ratio of distinct values to predetermine whether to apply dictionary encoding to the id columns [73]. When the number of time series to be managed increases, the metadata in the file will increase accordingly. Deserializing metadata results in more IO and CPU operations, which may reduce query performance. The time complexity of ORC (or Parquet) reading all metadata and scanning to locate the time series that needs to be read is $O(n)$. Since TsFile uses a tree-based structure to index metadata in the device, sensor, and time dimensions, the time complexity of locating the metadata of the target time series is $O(\log n)$. As shown in Figure 18(c), TsFile has the best read performance in the experiments.

TVStore [2] addresses the challenge of managing large volumes of time-series data by employing a time-varying, lossy compression approach. It compresses data iteratively at specified times and compression ratios, balancing storage limitations with data relevance based on age. MOST [71], a model-based lossy compression technique for time-series data, segments the data based on smooth changes and applies linear modeling to minimize storage, while outliers are stored separately for analytical accuracy. It uses a dual-mode query engine to process segments and outliers effectively. BUFF [44] introduces a precision-bounded, lossy compression for low-precision floating-point data. By utilizing data distribution characteristics like range and frequent values, BUFF achieves efficient encoding, allowing for in-situ adaptive queries and fast aggregation. Unlike TVStore and MOST, we focus on lossless compression of IoT data in this study. The experiments in Figure 19 of Section 8.2.4 show that our TsFile has lower storage cost than the lossless BUFF.

9.2 Time Series Database

Jensen et al. [35] provide a comprehensive survey of time series management systems. In addition to the time series native InfluxDB [30], RDBMS adapted TimescaleDB [65] and NoSQL adapted KairosDB [38] (OpenTSDB [48] also in this category), compared in Section 8.3, there are also other alternatives. The in-memory solutions, such as Gorilla [52] and Beringei [16] by Facebook,

ByteSeries [59] by ByteDance, or Monarch [1] by Google, are obviously not economic enough for the traditional manufacturing industry to persistent the whole historical data. On the other hand, model-based approaches, like MostDB [71], ModelarDB [36] and Plato [43], are often lossy (though efficient), and again do not meet the requirements of precise monitoring and control in industry. Thereby, our comparison focuses on some systems that have been widely used in real production.

Specifically, InfluxDB [30] is an open-source time series specialized database designed for application scenarios such as sensor monitoring and DevOps as well as real-time analytics. It has used the **Time Structured Merge (TSM)** tree as the storage engine since the release of v0.9.5 [33]. To avoid running out of memory when dealing with large query results, InfluxDB provides chunking [31] to return results as streamed batches instead of a single response. Although both InfluxDB and IoTDB use LSM-tree-based storage, the novel delay separation technique proposed by IoTDB better exploits the characteristics of IoT sensor data where usually only a small proportion of the data arrives late. Moreover, IoTDB has file-level statistics organized in a tree structure of devices and sensors for efficient access, while InfluxDB only has chunk-level maximum and minimum time information [33]. Therefore, as shown in Figure 21, IoTDB has better query performance than InfluxDB especially when dealing with large amounts of data.

TimescaleDB [65] is built on top of the relational database PostgreSQL. It is installed as a PostgreSQL extension tailored for time series and provides full SQL support. However, it also inherits the limitations of PostgreSQL. For example, the upper limit of the number of columns per table is 1,600 [56], which is not enough for IoT scenarios where there are thousands of devices with millions of sensors. TimescaleDB uses row-based storage, which is friendly to alignment queries. As shown in Figure 21(b), TimescaleDB's query performance is the second best when there are hundreds of series to be aligned. On the other hand, TimescaleDB processes data row by row, and thus the query cost is basically linear in the number of rows. It can be seen in Figure 21(a) and 21(c) that the query latency of TimescaleDB increases with the number of points.

KairosDB [38] is a distributed time series database run on the NoSQL database Cassandra. Multiple data points can be posted using the REST API [39], and the written batch of data is assembled into JSON format, whose parsing is time-consuming. In contrast, IoTDB uses the interface insertTablet() to improve write efficiency [7]. As illustrated in Figure 20, IoTDB has better write performance than KairosDB. Besides, KairosDB just puts the insertion into a local write buffer returns to the client, and then sends writes to its backend store Cassandra asynchronously, at the risk of write failure without noticing the client [61].

VergeDB [50] optimizes storage and transmission while allowing edge devices to perform analysis directly on compressed data. Similarly, as a columnar storage database, IoTDB also has advanced compression mechanisms to ensure the effectiveness on the edge. TransEdge [60], on the other hand, provides a secure and efficient read query protocol across untrusted edge nodes. Therefore, it focuses more on transactional capabilities rather than performance on the edge. In addition, SQLite [25] is widely adopted for lightweight, transactional storage in mobile and IoT devices, while DuckDB [57] is optimized for high-performance analytics on local data, making it ideal for data science and in-memory processing tasks on edge devices. Since VergeDB and TransEdge do not provide available code, the experiments in Figure 23 of Section 8.3.5 mainly demonstrate the superiority of our IoTDB compared to SQLite and DuckDB.

10 Conclusions

In this paper, we present a new time series management system, Apache IoTDB, with an open architecture specially designed to support both real-time query and big data analysis for IoT applications. The system includes a new time series file format, TsFile, with columnar storage of time and value to avoid null values and enable effective compression. Remarkably, without costly ETL,

the same TsFile can be written in end devices, queried in databases in the edge server, and analyzed in big data systems in the cloud. Built upon TsFile, the IoTDB engine uses an LSM-tree-like strategy to process extremely intensive writes and handle delayed data arrivals, very prevalent in IoT scenarios. A rich set of extensible queries, together with pre-computed statistics in TsFiles, enable both efficient OLTP and OLAP tasks in IoTDB. The results show that IoTDB can manage millions of time series, terabytes (TBs) of data, 10 million points insertion per second, 1-day data selection of 0.1 million points and 3-year data aggregation of over 10 million points in 100 milliseconds.

References

[1] Colin Adams, Luis Alonso, Benjamin Atkin, John Banning, Sumeer Bhola, Rick Buskens, Ming Chen, Xi Chen, Yoo Chung, Qin Jia, et al. 2020. Monarch: Google's planet-scale in-memory time series database. *Proc. VLDB Endow.* 13, 12 (2020), 3181–3194. DOI: https://doi.org/10.14778/3181-3194

[2] Yanzhe An, Yue Su, Yuqing Zhu, and Jianmin Wang. 2022. TVStore: Automatically bounding time series storage via time-varying compression. In *20th USENIX Conference on File and Storage Technologies, FAST 2022, Santa Clara, CA, USA, February 22–24, 2022*, Dean Hildebrand and Donald E. Porter (Eds.). USENIX Association, 83–100. https://www.usenix.org/conference/fast22/presentation/an

[3] Michael P. Andersen and David E. Culler. 2016. BTrDB: Optimizing storage system design for timeseries processing. In *14th USENIX Conference on File and Storage Technologies, FAST 2016, Santa Clara, CA, USA, February 22–25, 2016*, Angela Demke Brown and Florentina I. Popovici (Eds.). USENIX Association, 39–52. https://www.usenix.org/conference/fast16/technical-sessions/presentation/andersen

[4] Apache Hadoop. 2023. https://hadoop.apache.org/

[5] Apache HBase. 2023. http://hbase.apache.org/

[6] Apache IoTDB. 2023. https://iotdb.apache.org/

[7] Apache IoTDB. 2023. Data Manipulation Interface (DML Interface). https://iotdb.apache.org/UserGuide/V1.2.x/API/Programming-Java-Native-API.html#data-manipulation-interface-dml-interface

[8] Apache IoTDB. 2023. UDF Libraries. https://iotdb.apache.org/UserGuide/V1.2.x/Reference/UDF-Libraries.html

[9] Apache IoTDB. 2023. User-Defined Function (UDF). https://iotdb.apache.org/UserGuide/V1.2.x/User-Manual/Database-Programming.html#user-defined-function-udf

[10] Apache Kafka. 2023. https://kafka.apache.org/

[11] Apache Storm. 2023. http://storm.apache.org/

[12] Apache Zeppelin. 2023. https://zeppelin.apache.org/

[13] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational data processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31–June 4, 2015*, Timos K. Sellis, Susan B. Davidson, and Zachary G. Ives (Eds.). ACM, 1383–1394. DOI: https://doi.org/10.1145/2723372.2742797

[14] AVRO. 2023. https://avro.apache.org/docs/current/

[15] Francesco Beneventi, Andrea Bartolini, Carlo Cavazzoni, and Luca Benini. 2017. Continuous learning of HPC infrastructure models using big data analytics and in-memory processing tools. In *Design, Automation & Test in Europe Conference & Exhibition, DATE 2017, Lausanne, Switzerland, March 27–31, 2017*, David Atienza and Giorgio Di Natale (Eds.). IEEE, 1038–1043. DOI: https://doi.org/10.23919/DATE.2017.7927143

[16] Beringei. 2023. https://github.com/facebookarchive/beringei

[17] Yann Collet et al. 2013. LZ4: Extremely fast compression algorithm. *code. google. com* (2013).

[18] I. J. Cox. 1984. Increasing the bit packing densities of optical disk systems. *Applied Optics* 23 19 (1984), 3260–1.

[19] Jeffrey Dean and Sanjay Ghemawat. 2008. MapReduce: Simplified data processing on large clusters. *Commun. ACM* 51, 1 (2008), 107–113. DOI: https://doi.org/10.1145/1327452.1327492

[20] Peter Deutsch. 1996. GZIP file format specification version 4.3. *RFC* 1952 (1996), 1–12. DOI: https://doi.org/10.17487/RFC1952

[21] Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. 2018. IMPALA: Scalable distributed Deep-RL with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10–15, 2018 (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer G. Dy and Andreas Krause (Eds.). PMLR, 1406–1415. http://proceedings.mlr.press/v80/espeholt18a.html

[22] Chenguang Fang, Shaoxu Song, Haoquan Guan, Xiangdong Huang, Chen Wang, and Jianmin Wang. 2023. Grouping time series for efficient columnar storage. *Proc. ACM Manag. Data* 1, 1 (2023), 23:1–23:26. DOI: https://doi.org/10.1145/3588703

[23] Chenguang Fang, Shaoxu Song, and Yinan Mei. 2022. On repairing timestamps for regular interval time series. *Proc. VLDB Endow.* 15, 9 (2022), 1848–1860. DOI: https://doi.org/10.14778/3538598.3538607

[24] Chenguang Fang, Shaoxu Song, Yinan Mei, Ye Yuan, and Jianmin Wang. 2022. On aligning tuples for regression. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14–18, 2022*, Aidong Zhang and Huzefa Rangwala (Eds.). ACM, 336–346. DOI: https://doi.org/10.1145/3534678.3539373

[25] Kevin P. Gaffney, Martin Prammer, Laurence C. Brasfield, D. Richard Hipp, Dan R. Kennedy, and Jignesh M. Patel. 2022. SQLite: Past, present, and future. *Proc. VLDB Endow.* 15, 12 (2022), 3535–3547. DOI: https://doi.org/10.14778/3554821.3554842

[26] Solomon W. Golomb. 1966. Run-length encodings (Corresp.). *IEEE Trans. Inf. Theory* 12, 3 (1966), 399–401. DOI: https://doi.org/10.1109/TIT.1966.1053907

[27] Steinar H. Gunderson. 2023. Snappy: A fast compressor/decompressor. *code. google. com/p/snappy* (2023).

[28] Yongqiang He, Rubao Lee, Yin Huai, Zheng Shao, Namit Jain, Xiaodong Zhang, and Zhiwei Xu. 2011. RCFile: A fast and space-efficient data placement structure in MapReduce-based warehouse systems. In *Proceedings of the 27th International Conference on Data Engineering, ICDE 2011, April 11–16, 2011, Hannover, Germany*. IEEE Computer Society, 1199–1208. DOI: https://doi.org/10.1109/ICDE.2011.5767933

[29] Yin Huai, Ashutosh Chauhan, Alan Gates, Günther Hagleitner, Eric N. Hanson, Owen O'Malley, Jitendra Pandey, Yuan Yuan, Rubao Lee, and Xiaodong Zhang. 2014. Major technical advancements in Apache Hive. In *International Conference on Management of Data, SIGMOD 2014, Snowbird, UT, USA, June 22–27, 2014*, Curtis E. Dyreson, Feifei Li, and M. Tamer Özsu (Eds.). ACM, 1235–1246. DOI: https://doi.org/10.1145/2588555.2595630

[30] InfluxData. 2023. https://www.influxdata.com/time-series-platform/influxdb/

[31] InfluxData. 2023. Influxdb-Java Chunking. https://github.com/influxdata/influxdb-java/blob/master/MANUAL.md

[32] InfluxData. 2023. Influxdb-Java Client Library. https://github.com/influxdata/influxdb-java/

[33] InfluxData. 2023. The InfluxDB storage engine and the Time-Structured Merge Tree (TSM). https://archive.docs.influxdata.com/influxdb/v1.2/concepts/storage_engine/

[34] InfluxData. 2023. Retention Policy (RP). https://docs.influxdata.com/influxdb/v1.7/concepts/glossary/#retention-policy-rp/

[35] Søren Kejser Jensen, Torben Bach Pedersen, and Christian Thomsen. 2017. Time series management systems: A survey. *IEEE Trans. Knowl. Data Eng.* 29, 11 (2017), 2581–2600. DOI: https://doi.org/10.1109/TKDE.2017.2740932

[36] Søren Kejser Jensen, Torben Bach Pedersen, and Christian Thomsen. 2018. ModelarDB: Modular model-based time series management with Spark and Cassandra. *Proc. VLDB Endow.* 11, 11 (2018), 1688–1701. DOI: https://doi.org/10.14778/3236187.3236215

[37] Tian Jiang, Xiangdong Huang, Shaoxu Song, Chen Wang, Jianmin Wang, Ruibo Li, and Jincheng Sun. 2023. Non-blocking raft for high throughput IoT data. In *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3–7, 2023*. IEEE, 1140–1152. DOI: https://doi.org/10.1109/ICDE55515.2023.00092

[38] KairosDB. 2023. https://kairosdb.github.io/

[39] KairosDB. 2023. REST API. https://kairosdb.github.io/docs/restapi/AddDataPoints.html

[40] Yuyuan Kang, Xiangdong Huang, Shaoxu Song, Lingzhe Zhang, Jialin Qiao, Chen Wang, Jianmin Wang, and Julian Feinauer. 2022. Separation or not: On handing out-of-order time-series data in leveled LSM-Tree. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9–12, 2022*. IEEE, 3340–3352. DOI: https://doi.org/10.1109/ICDE53745.2022.00315

[41] Avinash Lakshman and Prashant Malik. 2010. Cassandra: A decentralized structured storage system. *ACM SIGOPS Oper. Syst. Rev.* 44, 2 (2010), 35–40. DOI: https://doi.org/10.1145/1773912.1773922

[42] Le-Le Li, Jiang-Yi Liu, Jianping Fan, Xuehai Qian, Kai Hwang, Yeh-Ching Chung, and Zhibin Yu. 2022. SOCA-DOM: A mobile system-on-chip array system for analyzing big data on the move. *J. Comput. Sci. Technol.* 37, 6 (2022), 1271–1289. DOI: https://doi.org/10.1007/s11390-022-1087-z

[43] Chunbin Lin, Etienne Boursier, and Yannis Papakonstantinou. 2020. Approximate analytics system over compressed time series with tight deterministic error guarantees. *Proc. VLDB Endow.* 13, 7 (2020), 1105–1118. DOI: https://doi.org/10.14778/3384345.3384357

[44] Chunwei Liu, Hao Jiang, John Paparrizos, and Aaron J. Elmore. 2021. Decomposed bounded floats for fast compression and queries. *Proc. VLDB Endow.* 14, 11 (2021), 2586–2598. DOI: https://doi.org/10.14778/3476249.3476305

[45] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2010. Dremel: Interactive analysis of web-scale datasets. *Proc. VLDB Endow.* 3, 1 (2010), 330–339. DOI: https://doi.org/10.14778/1920841.1920886

[46] Patrick E. O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth J. O'Neil. 1996. The log-structured merge-tree (LSM-Tree). *Acta Informatica* 33, 4 (1996), 351–385. DOI: https://doi.org/10.1007/s002360050048

[47] Diego Ongaro and John Ousterhout. 2014. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (Usenix ATC'14)*. 305–319.

[48] OpenTSDB. 2023. http://opentsdb.net/
[49] Apache ORC. 2023. https://orc.apache.org/
[50] John Paparrizos, Chunwei Liu, Bruno Barbarioli, Johnny Hwang, Ikraduya Edian, Aaron J. Elmore, Michael J. Franklin, and Sanjay Krishnan. 2021. VergeDB: A database for IoT analytics on edge devices. In *11th Conference on Innovative Data Systems Research, CIDR 2021, Virtual Event, January 11–15, 2021, Online Proceedings*. www.cidrdb.org. http://cidrdb.org/cidr2021/papers/cidr2021_paper11.pdf
[51] Apache Parquet. 2023. https://parquet.apache.org/
[52] Tuomas Pelkonen, Scott Franklin, Paul Cavallaro, Qi Huang, Justin Meza, Justin Teller, and Kaushik Veeraraghavan. 2015. Gorilla: A fast, scalable, in-memory time series database. *Proc. VLDB Endow.* 8, 12 (2015), 1816–1827. DOI: https://doi.org/10.14778/2824032.2824078
[53] Tim Peters. 2002. Timsort Description. https://svn.python.org/projects/python/trunk/Objects/listsort.txt
[54] PI. 2023. https://docs.osisoft.com/bundle/pi-server/page/the-structure-of-pi-af-asset-models_2.html
[55] PostgreSQL. 2023. https://www.postgresql.org/
[56] PostgreSQL. 2023. PostgreSQL Limits. https://www.postgresql.org/docs/current/limits.html
[57] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30–July 5, 2019*, Peter A. Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). ACM, 1981–1984. DOI: https://doi.org/10.1145/3299869.3320212
[58] Dennis G. Severance and Guy M. Lohman. 1976. Differential files: Their application to the maintenance of large databases. *ACM Trans. Database Syst.* 1, 3 (1976), 256–267. DOI: https://doi.org/10.1145/320473.320484
[59] Xuanhua Shi, Zezhao Feng, Kaixi Li, Yongluan Zhou, Hai Jin, Yan Jiang, Bingsheng He, Zhijun Ling, and Xin Li. 2020. ByteSeries: An in-memory time series database for large-scale monitoring systems. In *SoCC '20: ACM Symposium on Cloud Computing, Virtual Event, USA, October 19–21, 2020*, Rodrigo Fonseca, Christina Delimitrou, and Beng Chin Ooi (Eds.). ACM, 60–73. DOI: https://doi.org/10.1145/3419111.3421289
[60] Abhishek A. Singh, Aasim Khan, Sharad Mehrotra, and Faisal Nawab. 2023. TransEdge: Supporting efficient read queries across untrusted edge nodes. In *Proceedings 26th International Conference on Extending Database Technology, EDBT 2023, Ioannina, Greece, March 28–31, 2023*, Julia Stoyanovich, Jens Teubner, Nikos Mamoulis, Evaggelia Pitoura, Jan Mühlig, Katja Hose, Sourav S. Bhowmick, and Matteo Lissandrini (Eds.). OpenProceedings.org, 684–696. DOI: https://doi.org/10.48786/EDBT.2023.57
[61] Aleksandar Stojadinovic. 2023. Write failures caused by overload not reported. https://github.com/kairosdb/kairosdb/issues/145
[62] Yunxiang Su, Wenxuan Ma, and Shaoxu Song. 2023. Learning autoregressive model in LSM-tree based store. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6–10, 2023*, Ambuj K. Singh, Yizhou Sun, Leman Akoglu, Dimitrios Gunopulos, Xifeng Yan, Ravi Kumar, Fatma Ozcan, and Jieping Ye (Eds.). ACM, 2061–2071. DOI: https://doi.org/10.1145/3580305.3599405
[63] Yu Sun, Shaoxu Song, Chen Wang, and Jianmin Wang. 2020. Swapping repair for misplaced attribute values. In *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20–24, 2020*. IEEE, 721–732. DOI: https://doi.org/10.1109/ICDE48307.2020.00068
[64] Yu Sun, Zheng Zheng, Shaoxu Song, and Fei Chiang. 2022. Confidence bounded replica currency estimation. In *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12–17, 2022*, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.). ACM, 730–743. DOI: https://doi.org/10.1145/3514221.3517852
[65] TimescaleDB. 2023. https://www.timescale.com/
[66] TimescaleDB. 2023. Time Series Benchmark Suite (TSBS). https://github.com/timescale/tsbs
[67] Chen Wang, Jialin Qiao, Xiangdong Huang, Shaoxu Song, Haonan Hou, Tian Jiang, Lei Rui, Jianmin Wang, and Jiaguang Sun. 2023. Apache IoTDB: A time series database for IoT applications. *Proc. ACM Manag. Data* 1, 2 (2023), 195:1–195:27. DOI: https://doi.org/10.1145/3589775
[68] Haoyu Wang and Shaoxu Song. 2022. Frequency domain data encoding in Apache IoTDB. *Proc. VLDB Endow.* 16, 2 (2022), 282–290. DOI: https://doi.org/10.14778/3565816.3565829
[69] Jinzhao Xiao, Yuxiang Huang, Changyu Hu, Shaoxu Song, Xiangdong Huang, and Jianmin Wang. 2022. Time series data encoding for efficient storage: A comparative analysis in Apache IoTDB. *Proc. VLDB Endow.* 15, 10 (2022), 2148–2160. DOI: https://doi.org/10.14778/3547305.3547319
[70] Fangjin Yang, Eric Tschetter, Xavier Léauté, Nelson Ray, Gian Merlino, and Deep Ganguli. 2014. Druid: A real-time analytical data store. In *International Conference on Management of Data, SIGMOD 2014, Snowbird, UT, USA, June 22–27, 2014*, Curtis E. Dyreson, Feifei Li, and M. Tamer Özsu (Eds.). ACM, 157–168. DOI: https://doi.org/10.1145/2588555.2595631
[71] Zehai Yang and Shimin Chen. 2023. MOST: Model-based compression with outlier storage for time series data. *Proc. ACM Manag. Data* 1, 4 (2023), 250:1–250:29. DOI: https://doi.org/10.1145/3626737

[72] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2010. Spark: Cluster computing with working sets. In *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing* (Boston, MA) *(HotCloud'10)*. USENIX Association, USA, 10.

[73] Xinyu Zeng, Yulong Hui, Jiahong Shen, Andrew Pavlo, Wes McKinney, and Huanchen Zhang. 2023. An empirical evaluation of columnar storage formats. *Proc. VLDB Endow.* 17, 2 (2023), 148–161. https://www.vldb.org/pvldb/vol17/p148-zeng.pdf

[74] Xiaojian Zhang, Hongyin Zhang, Shaoxu Song, Xiangdong Huang, Chen Wang, and Jianmin Wang. 2023. Backward-sort for time series in Apache IoTDB. In *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3–7, 2023*. IEEE, 3196–3208. DOI: https://doi.org/10.1109/ICDE55515.2023.00245

Received 23 December 2023; revised 1 November 2024; accepted 9 March 2025